AWS AI Security Agent

Code Review Report

JuiceShop-Code

August 14, 2026

Table of Contents

Report Filters Applied 6

Executive Summary 8

Scope 10

Methodology 11

Findings (72) 12

Detailed Findings 17

Security Findings

Remote Code Execution via YAML Deserialization (Unauthenticated) 17

JWT Algorithm Confusion Attack via Missing Algorithm Enforcement 18

Server-Side Code Execution via eval() on Username in Profile Rendering 19

SQL Injection in Login Endpoint via Email Parameter 20

Hardcoded JWT Private Key and HMAC Secret Enable Token Forgery 21

TLS/JWT Private Key Exposed via Unauthenticated /infrastructure Directory Browsing 23

Remote Code Execution via B2B Order Endpoint (notevil sandbox escape) 25

Privilege Escalation via Role Mass Assignment in User Registration 26

NoSQL Injection in Track Order Endpoint via $where Clause 27

Server-Side Request Forgery via Profile Image URL Upload 28

XML External Entity (XXE) Injection via File Upload Enables File Read and SSRF 29

Unsalted MD5 Password Hashing Enables Trivial Hash Reversal 30

HTTP ALB Listener Forwards Traffic Without TLS Redirect 31

Password Hash and TOTP Secret Leaked in JWT Payload 32

Missing Authorization on Product Update Endpoint 33

NoSQL Injection in Review Update Enables Mass Modification **34**

Stored XSS via User Email Registration Targeting Admin Panel via bypassSecurityTrustHtml **35**

Predictable OAuth User Password Derived from Reversed Email Enables Account Takeover **37**

Unrestricted CORS Policy Allows Cross-Origin Data Theft **39**

Null Byte Injection Bypasses File Extension Allowlist in FTP Server **40**

Password Hash Leak and JSONP Cross-Origin Data Theft via whoami Endpoint **41**

Unauthenticated Access to Server Log Files Containing Sensitive Data **43**

Wallet Balance Manipulation via Missing Positive-Value Validation **44**

Zip Slip Path Traversal - Arbitrary File Write (Unauthenticated) **45**

CSRF Password Change via GET Request Without Current Password Verification **46**

SQL Injection in Product Search via Query Parameter **47**

Stored XSS via Unauthenticated Product Description Modification **48**

Unauthenticated Access to JWT Public Key Enabling Algorithm Confusion Attack **49**

2FA Bypass via Forged tmpToken and Rate Limit Bypass **50**

Credit Card Numbers Stored in Plaintext Without Encryption **51**

Default Admin Credentials Enable Unauthenticated Administrative Access **52**

Unauthenticated Exposure of Security Question Answers Enabling Account Takeover **53**

IDOR - Checkout Any User's Basket Without Ownership Verification **54**

Local File Inclusion via Layout Parameter in Data Erasure Endpoint **55**

Hardcoded Ethereum Mnemonic Phrase in Source Code **56**

Rate Limit Bypass via X-Forwarded-For Header Spoofing **57**

IDOR in PUT /api/Addresss/:id Allows Modifying Any User's Address **58**

Coupon Forgery via Trivial z85 Encoding Without Cryptographic Signature **59**

IDOR - Any Authenticated User Can Access Any Basket **60**

Forged Feedback UserId Allows Impersonation **61**

Open Redirect via Substring-Based Allowlist Bypass **62**

Captcha Answer Leaked in API Response Enables Automated Bypass **63**

LLM Prompt Injection Bypasses Coupon Policy for Arbitrary Discount Generation **64**

Stored XSS in Feedback via sanitize-html v1.4.2 Bypass **65**

Review Author Spoofing via Unverified Author Field **66**

Missing Authorization Check on Review Update (Any User Can Edit Any Review) **67**

Hardcoded Test Credentials Exposed in Angular Frontend Bundle **68**

JWT Token Stored in localStorage and Insecure Cookie Enabling Session Theft via XSS **69**

Reflected XSS via Track Order Response Rendered with bypassSecurityTrustHtml **71**

Unauthenticated Order Tracking Reveals Sensitive Order Details **72**

Unauthenticated Full Application Configuration Exposure **73**

Authenticated User Enumeration Exposes All User Emails, Roles, and IPs Without Admin Restriction **74**

Verbose Error Handler Exposes Stack Traces in Production **76**

LLM Tool Call Arguments Including Coupon Codes Leaked via SSE Stream **77**

Stored XSS via True-Client-IP Header in Login IP Storage **78**

CSP Header Injection via User-Controlled profileImage Field **79**

NoSQL Injection via $where in Product Reviews Listing **80**

Passwords Exposed in URL Query String via GET-Based Password Change **81**

No Rate Limiting on Login Endpoint Enables Brute Force **82**

Denial of Service via XML Entity Expansion and YAML Bomb **83**

Non-Compliant Requirements

[Requirement: authentication/JWT/crypto] Hardcoded JWT Private Key with Algorithm Confusion Enables Token Forgery **84**

[Requirement: authorization-best-practices] IDOR on Basket Access and Missing Authorization on Product Modification **87**

[Requirement: information-protection] Credit Card Numbers Stored as Plaintext Integers Extractable via SQL Injection **89**

[Requirement: secure-by-default] Insecure Default Configuration: Permissive CORS, No CSP, Insecure Cookies, Public Directories **92**

[Requirement: log-protection] Logs Publicly Accessible Without Authentication Containing Sensitive Data **95**

[Requirement: privileged-access] Role Injection in User Registration Enables Admin Self-Enrollment Without Guardrails **97**

[Requirement: log-protection/information-protection] Password Change via GET Logged by Morgan and Served Publicly Creates Credential Exposure Chain **99**

[Requirement: information-protection/secure-by-default] HTTP ALB Listener Forwards Traffic Without HTTPS Redirect **101**

[Requirement: file-hosting/information-protection/tenant-isolation] Unauthenticated FTP-Style File Hosting Exposes Customer PII and Secrets **104**

[Requirement: audit-logging/privileged-access] Complete Absence of Security Audit Logging Across All Security-Critical Operations **107**

[Requirement: authentication/secure-by-default] No JWT Token Revocation Enables Persistent Access After Password Change **110**

[Requirement: JWT/authorization/tenant-isolation] Chatbot Uses Unverified JWT Decode Enabling Cross-User Order Data Access **113**

Report Filters Applied

This report includes the following filters:

Risk levels:

Critical | High | Medium | Low

Confidence:

High

Status:

Active | Accepted | Resolved

Risk types:

Access Control:

Broken Object Property Authorization | Insecure Direct Object Reference | Local File Inclusion | Path Traversal

Authentication & Authorization:

Authentication Bypass | Default Credentials | Json Web Token Vulnerabilities | Privilege Escalation | Session Token Vulnerabilities

Business Logic:

Business Logic Vulnerabilities | Information Disclosure

Client-Side:

Cross Site Scripting

Configuration & Crypto:

Cryptographic Vulnerabilities | Security Misconfiguration

Injection Attacks:

Code Injection | Sql Injection | Xml External Entity

Network:

Server Side Request Forgery

Resource & Performance:

Denial Of Service

Task status

Finding types

Security Findings | Non-Compliant Requirements

Executive Summary

This code review report summarizes findings for **JuiceShop-Code** as of August 13, 2026. **72** reported security findings were identified, comprising **9 critical-severity**, **35 high-severity**, **28 medium-severity**, **0 low-severity**, and **0 informational-severity** issues. Critical and high-severity findings include **Remote Code Execution via YAML Deserialization (Unauthenticated)**, **JWT Algorithm Confusion Attack via Missing Algorithm Enforcement**, and **42** more. Full details for all findings are available in the Findings section.

Understanding Finding Metrics

Agent confidence level:

Indicates AWS Security Agent's confidence that a finding represents a genuine security vulnerability. For code-scanner findings, confidence reflects how much of the evidence walk from an attacker-triggerable entry point to the defective code was verified by reading this repository.

High confidence findings have every hop of the walk backed by code read in this repository.

Medium confidence findings have one link that depends on a named external behavior the scanner cannot read here — a template engine, a downstream service, a framework default, or a deployment configuration. Review whether that external behavior holds in your environment.

Low confidence findings describe defective code where the scanner could not close a link even in this repository. Treat as leads that warrant manual investigation.

Severity levels:

- **Critical** Requires immediate action; exploitation could lead to system compromise.
- **High** Requires prompt attention; exploitation could result in significant security impact.
- **Medium** Should be addressed in a reasonable timeframe; contributes to overall security risk.
- **Low** Can be addressed as part of regular maintenance; minimal immediate risk.
- **Informational** For informational purposes; minimal to no immediate risk.

Risk score: A numerical assessment combining severity with the likelihood of exploitation. Code-scanner findings currently report severity only; CVSS v3.1 scoring is in development and will appear alongside severity once it lands.

Finding status: Tracks the remediation workflow state. Active findings require review. Mark findings as Resolved when fixed, Accepted when risk is acknowledged, or False Positive when determined to be incorrect. Incomplete findings require additional investigation.

Scope

This code review analyzed the following assets and configuration.

Review type

- Full scan

Source Code:

- s3://juiceshop-ecs-securityagentresources-oz17q80ztcfr/juice-shop-master.zip

Methodology

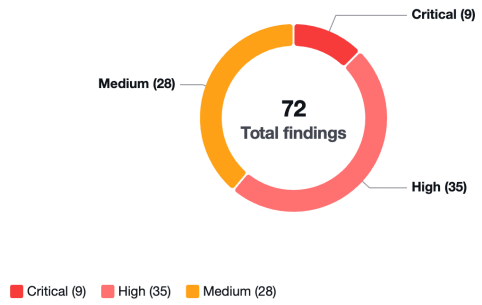
This code review was conducted using AWS Security Agent, which deploys specialized AI agents through a three-phase approach: Preflight (set up logging infrastructure and environment), Static Analysis (code and configuration review), and Finalizing (validation and reporting). The agent analyzes application context from source code and documentation to identify vulnerabilities.

Testing Approach

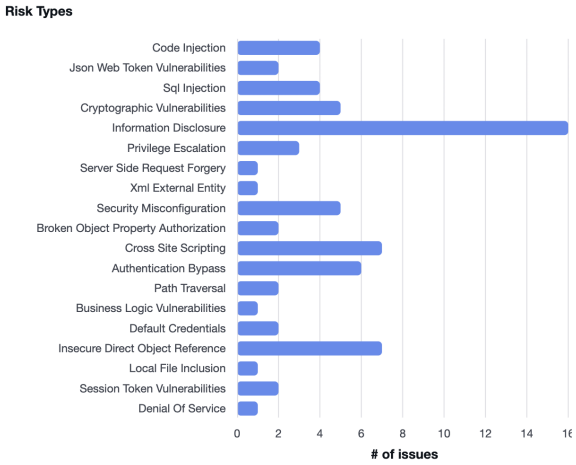
AWS Security Agent performs static analysis on the repository's source code and configuration to surface security vulnerabilities. Each finding includes a severity assessment, a confidence rating, and the specific code locations involved. The analysis follows industry-standard patterns while adapting to application-specific context.

Findings (72)

Severity Distribution



Risk Types



Security Findings (60)

Finding	Severity	Status
Remote Code Execution via YAML Deserialization (Unauthenticated)	Critical	Active
JWT Algorithm Confusion Attack via Missing Algorithm Enforcement	Critical	Active
Server-Side Code Execution via eval() on Username in Profile Rendering	Critical	Active
SQL Injection in Login Endpoint via Email Parameter	Critical	Active
Hardcoded JWT Private Key and HMAC Secret Enable Token Forgery	Critical	Active
TLS/JWT Private Key Exposed via Unauthenticated /infrastructure Directory Browsing	Critical	Active
Remote Code Execution via B2B Order Endpoint (notevil sandbox escape)	Critical	Active
Privilege Escalation via Role Mass Assignment in User Registration	Critical	Active
NoSQL Injection in Track Order Endpoint via $where Clause	High	Active
Server-Side Request Forgery via Profile Image URL Upload	High	Active

XML External Entity (XXE) Injection via File Upload Enables File Read and SSRF	High	Active
Unsalted MD5 Password Hashing Enables Trivial Hash Reversal	High	Active
HTTP ALB Listener Forwards Traffic Without TLS Redirect	High	Active
Password Hash and TOTP Secret Leaked in JWT Payload	High	Active
Missing Authorization on Product Update Endpoint	High	Active
NoSQL Injection in Review Update Enables Mass Modification	High	Active
Stored XSS via User Email Registration Targeting Admin Panel via bypassSecurityTrustHtml	High	Active
Predictable OAuth User Password Derived from Reversed Email Enables Account Takeover	High	Active
Unrestricted CORS Policy Allows Cross-Origin Data Theft	High	Active
Null Byte Injection Bypasses File Extension Allowlist in FTP Server	High	Active
Password Hash Leak and JSONP Cross-Origin Data Theft via whoami Endpoint	High	Active
Unauthenticated Access to Server Log Files Containing Sensitive Data	High	Active
Wallet Balance Manipulation via Missing Positive-Value Validation	High	Active
Zip Slip Path Traversal - Arbitrary File Write (Unauthenticated)	High	Active
CSRF Password Change via GET Request Without Current Password Verification	High	Active
SQL Injection in Product Search via Query Parameter	High	Active
Stored XSS via Unauthenticated Product Description Modification	High	Active
Unauthenticated Access to JWT Public Key Enabling Algorithm Confusion Attack	High	Active

2FA Bypass via Forged tmpToken and Rate Limit Bypass	High	Active
Credit Card Numbers Stored in Plaintext Without Encryption	High	Active
Default Admin Credentials Enable Unauthenticated Administrative Access	High	Active
Unauthenticated Exposure of Security Question Answers Enabling Account Takeover	High	Active
IDOR - Checkout Any User's Basket Without Ownership Verification	High	Active
Local File Inclusion via Layout Parameter in Data Erasure Endpoint	High	Active
Hardcoded Ethereum Mnemonic Phrase in Source Code	High	Active
Rate Limit Bypass via X-Forwarded-For Header Spoofing	Medium	Active
IDOR in PUT /api/Addresss/:id Allows Modifying Any User's Address	Medium	Active
Coupon Forgery via Trivial z85 Encoding Without Cryptographic Signature	Medium	Active
IDOR - Any Authenticated User Can Access Any Basket	Medium	Active
Forged Feedback UserId Allows Impersonation	Medium	Active
Open Redirect via Substring-Based Allowlist Bypass	Medium	Active
Captcha Answer Leaked in API Response Enables Automated Bypass	Medium	Active
LLM Prompt Injection Bypasses Coupon Policy for Arbitrary Discount Generation	Medium	Active
Stored XSS in Feedback via sanitize-html v1.4.2 Bypass	Medium	Active
Review Author Spoofing via Unverified Author Field	Medium	Active
Missing Authorization Check on Review Update (Any User Can Edit Any Review)	Medium	Active

Hardcoded Test Credentials Exposed in Angular Frontend Bundle	Medium	Active
JWT Token Stored in localStorage and Insecure Cookie Enabling Session Theft via XSS	Medium	Active
Reflected XSS via Track Order Response Rendered with bypassSecurityTrustHtml	Medium	Active
Unauthenticated Order Tracking Reveals Sensitive Order Details	Medium	Active
Unauthenticated Full Application Configuration Exposure	Medium	Active
Authenticated User Enumeration Exposes All User Emails, Roles, and IPs Without Admin Restriction	Medium	Active
Verbose Error Handler Exposes Stack Traces in Production	Medium	Active
LLM Tool Call Arguments Including Coupon Codes Leaked via SSE Stream	Medium	Active
Stored XSS via True-Client-IP Header in Login IP Storage	Medium	Active
CSP Header Injection via User-Controlled profileImage Field	Medium	Active
NoSQL Injection via $where in Product Reviews Listing	Medium	Active
Passwords Exposed in URL Query String via GET-Based Password Change	Medium	Active
No Rate Limiting on Login Endpoint Enables Brute Force	Medium	Active
Denial of Service via XML Entity Expansion and YAML Bomb	Medium	Active

Non-Compliant Requirements (12)

Finding	Severity	Status
[Requirement: authentication/JWT/crypto] Hardcoded JWT Private Key with Algorithm Confusion Enables Token Forgery	Critical	Active

[Requirement: authorization-best-practices] IDOR on Basket Access and Missing Authorization on Product Modification	High	Active
[Requirement: information-protection] Credit Card Numbers Stored as Plaintext Integers Extractable via SQL Injection	High	Active
[Requirement: secure-by-default] Insecure Default Configuration: Permissive CORS, No CSP, Insecure Cookies, Public Directories	High	Active
[Requirement: log-protection] Logs Publicly Accessible Without Authentication Containing Sensitive Data	High	Active
[Requirement: privileged-access] Role Injection in User Registration Enables Admin Self-Enrollment Without Guardrails	High	Active
[Requirement: log-protection/information-protection] Password Change via GET Logged by Morgan and Served Publicly Creates Credential Exposure Chain	High	Active
[Requirement: information-protection/secure-by-default] HTTP ALB Listener Forwards Traffic Without HTTPS Redirect	High	Active
[Requirement: file-hosting/information-protection/tenant-isolation] Unauthenticated FTP-Style File Hosting Exposes Customer PII and Secrets	High	Active
[Requirement: audit-logging/privileged-access] Complete Absence of Security Audit Logging Across All Security-Critical Operations	Medium	Active
[Requirement: authentication/secure-by-default] No JWT Token Revocation Enables Persistent Access After Password Change	Medium	Active
[Requirement: JWT/authorization/tenant-isolation] Chatbot Uses Unverified JWT Decode Enabling Cross-User Order Data Access	Medium	Active

Finding 1: Remote Code Execution via YAML Deserialization (Unauthenticated)

ID	**f-1c68c801-232c-4e44-abfa-e05d21481ce4**
Severity	**Critical**
Risk Type	**Code Injection**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:24 PM (GMT+7)**

Description

YAML file upload processing uses js-yaml's `yaml.load()` function (unsafe by default) in a vm context. The `!!js/function` tag in js-yaml allows code execution when loading YAML.

An unauthenticated attacker can achieve remote code execution by uploading a crafted YAML file with embedded JavaScript functions.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/routes/fileUpload.ts (L102) — sink: YAML deserialization with code execution
```

Evidence

routes/fileUpload.ts:102:

const yamlString = vm.runInContext('JSON.stringify(yaml.load(data))', sandbox, { timeout: 2000 })

// yaml.load with !!js/function enables code execution

Suggested fix

Use yaml.safeLoad() instead of yaml.load(). Or use js-yaml with schema: yaml.SAFE_SCHEMA.

Finding 2: JWT Algorithm Confusion Attack via Missing Algorithm Enforcement

ID	**f-6fdb8937-928b-41ec-9155-5ab89a6efbe7**
Severity	**Critical**
Risk Type	**Json Web Token Vulnerabilities**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:19 PM (GMT+7)**

Description

The express-jwt middleware is configured without specifying allowed algorithms: `expressJwt({ secret: [REDACTED] })`. With the old express-jwt@0.1.3 version, this allows algorithm confusion attacks where an attacker can sign a token with HS256 using the public key as the HMAC secret.

An attacker can forge valid JWT tokens by signing with the publicly available RSA public key using HS256 algorithm.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/lib/insecurity.ts (L52) — sink: express-jwt without algorithm restriction
```

Evidence

lib/insecurity.ts:52:

export const isAuthorized = () => expressJwt(({ secret: [REDACTED] }) as any)

// No algorithms array specified - accepts any algorithm including HS256

Suggested fix

Specify algorithms: expressJwt({ secret: [REDACTED], algorithms: ['RS256'] })

Finding 3: Server-Side Code Execution via eval() on Username in Profile Rendering

ID	**f-44fb4e90-dfd0-40cd-983c-2a23eca2a5af**
Severity	**Critical**
Risk Type	**Code Injection**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:18 PM (GMT+7)**

Description

The user profile rendering function uses eval() on username content matching a `#{...}` pattern. A user who sets their username to a pattern like `#{require('child_process').execSync('id')}` can achieve server-side code execution when their profile is viewed.

An attacker can execute arbitrary JavaScript on the server by setting a crafted username, leading to full server compromise.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/routes/userProfile.ts (L63–L68) — sink: eval() on username content
```

Evidence

routes/userProfile.ts:63-68:

```
if (username?.match(/#{(.*)}/) !== null) {
 const code = username?.substring(2, username.length - 1)
 username = eval(code) // eslint-disable-line no-eval
}
```

Suggested fix

Remove eval() usage entirely. Render usernames as plain text without interpreting embedded expressions.

Finding 4: SQL Injection in Login Endpoint via Email Parameter

ID	**f-6b4e7398-d508-48d0-9e7d-60c80dd58c29**
Severity	**Critical**
Risk Type	**Sql Injection**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:17 PM (GMT+7)**

Description

The login endpoint constructs a raw SQL query by directly interpolating user-supplied email into the query string without parameterization.

An attacker can bypass authentication entirely, log in as any user (including admin), or extract the entire database contents via UNION-based injection.

Code locations

`/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/routes/login.ts (L32) — sink: raw SQL query with string interpolation of req.body.email`

Evidence

routes/login.ts:32:

models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email || ''}' AND password: [REDACTED] || '')}' AND deletedAt IS NULL`, { model: UserModel, plain: true })

Suggested fix

Use parameterized queries: models.sequelize.query('SELECT * FROM Users WHERE email = ? AND password: [REDACTED] AND deletedAt IS NULL', { replacements: [req.body.email, security.hash(req.body.password)], model: UserModel, plain: true })

Finding 5: Hardcoded JWT Private Key and HMAC Secret Enable Token Forgery

ID	**f-1769c56b-497c-4b19-aaeb-be4a1c0c1faa**
Severity	**Critical**
Risk Type	**Cryptographic Vulnerabilities**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:19 PM (GMT+7)**

Description

The JWT RSA private key is hardcoded directly in the source code (lib/insecurity.ts:21), and the HMAC secret 'pa4qacea4VK9t9nGv7yZtwmj' is also hardcoded (lib/insecurity.ts:42). Anyone with source code access can forge JWT tokens for any user and generate valid HMAC-based tokens (deluxe tokens, security answer hashes).

An attacker can forge authentication tokens for any user including admin, bypass 2FA via forged tmpTokens, and impersonate any user in the system.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/lib/insecurity.ts (L21) — sink: hardcoded RSA private key for JWT signing
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/lib/insecurity.ts (L42) — sink: hardcoded HMAC secret
```

Evidence

lib/insecurity.ts:21:

const privateKey: [REDACTED] RSA PRIVATE KEY-----\r\nMIICXAIBAAKBgQDNwqLEe9wg...'

lib/insecurity.ts:42:

export const hmac = (data: string) => crypto.createHmac('sha256', 'pa4qacea4VK9t9nGv7yZtwmj').up-
date(data).digest('hex')

Suggested fix

Store keys in environment variables or a secrets management system. Rotate keys regularly. Never commit private keys to source control.

Finding 6: TLS/JWT Private Key Exposed via Unauthenticated /infrastructure Directory Browsing

ID	**f-43d310ae-0407-4737-96ef-4fa970ad39bf**
Severity	**Critical**
Risk Type	**Information Disclosure**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:17 PM (GMT+7)**

Description

The `/infrastructure` directory is served with directory browsing enabled and no authentication at server.ts:253-267. When a file ends with `.tf`, `.yml`, or `Dockerfile`, it is read and served as text/plain (server.ts:260-264). The Terraform file `infrastructure/terraform/networking.tf` at line 171 contains a hardcoded RSA private key (the same one used for JWT signing at lib/insecurity.ts:22-28). This means any unauthenticated user can browse to `/infrastructure/terraform/networking.tf` and obtain the TLS/JWT private key.

An attacker can request `GET /infrastructure/terraform/networking.tf` without authentication to obtain the RSA private key. This key is the same one used for JWT signing (lib/insecurity.ts:22-28), enabling complete authentication bypass via token forgery. It's also used as the TLS server certificate private key, enabling MITM attacks against HTTPS connections.

Verified: server.ts:253 enables directory browsing at `/infrastructure` with no auth. server.ts:256-264 serves .tf files as text/plain. The `verify.accessControlChallenges()` at line 254 is for challenge verification only (not access control). The networking.tf file at line 171 contains the full private key. The vuln-code-snippet comments are filtered (line 263) but the key itself remains.

Could not verify: Whether a reverse proxy blocks access to /infrastructure in production. The code explicitly serves it.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/server.ts (L253–L267) — sink: unauthenticated infrastructure directory listing and file serving
```

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/in-
frastructure/terraform/networking.tf (L171) — source: hardcoded TLS/JWT private key in
Terraform config
```

Evidence

server.ts:253-267:

app.use('/infrastructure', serveIndexMiddleware, serveIndex('infrastructure', { icons: true, view: 'details', ... }))

app.use('/infrastructure', verify.accessControlChallenges()) // NOT access control - just challenge tracking

app.use('/infrastructure', (req, res, next) => {

 const filePath = path.resolve('infrastructure', path.normalize(req.path)...)

 if (filePath.endsWith('.tf') || filePath.endsWith('.yml') || filePath.endsWith('Dockerfile')) {

 fs.readFile(filePath, 'utf8', (err, data) => {

 const cleaned = data.split('\n').filter(line => !line.trim().match(/^#\s*vuln-code-snippet\s/))...

 res.type('text/plain').send(cleaned) // Serves TF files with private key

})

}

})

infrastructure/terraform/networking.tf:171:

private_key: [REDACTED] RSA PRIVATE KEY-----\nMIICXAIBAAKBgQDNwqLEe9wg..."

// Same key as lib/insecurity.ts:22-28 (JWT signing key)

Suggested fix

Remove the /infrastructure directory from the web application's served paths entirely.

1. At server.ts:253-267, remove the entire /infrastructure directory browsing and file serving block. Infrastructure-as-code files should never be served by the application.
2. Move infrastructure files out of the application's deployment artifact or add them to .dockerignore/.npmignore.
3. The private key at networking.tf:171 must be rotated immediately as it is publicly accessible. Use AWS Certificate Manager (ACM) instead of hardcoded keys.

Finding 7: Remote Code Execution via B2B Order Endpoint (notevil sandbox escape)

ID	**f-ce05e6f4-6a40-4029-9c99-2ad699ae8271**
Severity	**Critical**
Risk Type	**Code Injection**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:18 PM (GMT+7)**

Description

The B2B order endpoint evaluates user-supplied orderLinesData in a vm sandbox using `notevil` library's safeEval. The notevil library has known sandbox escape vulnerabilities allowing arbitrary code execution on the server.

An attacker with any authenticated account can achieve full remote code execution on the server, compromising all data and gaining OS-level access.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/routes/b2bOrder.ts (L20–L23) — sink: vm sandbox with notevil safeEval evaluates user input
```

Evidence

routes/b2bOrder.ts:20-23:

const sandbox = { safeEval, orderLinesData }

vm.createContext(sandbox)

vm.runInContext('safeEval(orderLinesData)', sandbox, { timeout: 2000 })

Suggested fix

Remove eval-based processing entirely. Parse order data as structured JSON instead of evaluating it as code.

Finding 8: Privilege Escalation via Role Mass Assignment in User Registration

ID	**f-823db95c-908d-4281-8b84-dda00478e260**
Severity	**Critical**
Risk Type	**Privilege Escalation**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:19 PM (GMT+7)**

Description

User registration at POST /api/Users accepts a 'role' field in the request body, allowing any user to self-register as admin by including `\"role\": \"admin\"` in the registration request.

An attacker can create admin accounts without any authorization, gaining full administrative access to the application.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/server.ts (L427–L439) — sink: user registration endpoint allows role field in body
```

Evidence

server.ts:427-439 registers User model CRUD. The UserModel allows setting 'role' via mass assignment, with no server-side enforcement preventing role elevation during registration.

Suggested fix

Filter allowed fields on registration. Never allow role to be set via user input. Use an allowlist of permitted registration fields.

Finding 9: NoSQL Injection in Track Order Endpoint via $where Clause

ID	**f-70af25c1-bcaa-49c4-84ba-0c5d6f030f7f**
Severity	**High**
Risk Type	**Sql Injection**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:18 PM (GMT+7)**

Description

The track order endpoint uses MongoDB's $where operator with string interpolation of user-supplied orderId, enabling NoSQL injection to access or manipulate orders.

An attacker can bypass order ID matching to enumerate all orders, or execute arbitrary JavaScript in the MongoDB context.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/routes/trackOrder.ts (L17) — sink: MongoDB $where with string interpolation
```

Evidence

routes/trackOrder.ts:17:

db.ordersCollection.find({ $where: `this.orderId === '${id}'` })

Suggested fix

Use standard MongoDB query operators: db.ordersCollection.find({ orderId: id })

Finding 10: Server-Side Request Forgery via Profile Image URL Upload

ID	**f-7d81b56c-e6c3-43a4-9a2f-5909b00d6f66**
Severity	**High**
Risk Type	**Server Side Request Forgery**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:18 PM (GMT+7)**

Description

The profile image URL upload endpoint fetches any URL provided by the user via `fetch(url)` without validating the target. An attacker can specify internal URLs (e.g., cloud metadata, internal services) to perform SSRF.

An attacker can access internal services, cloud metadata endpoints (169.254.169.254), and other resources not directly reachable from the internet.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/routes/profileImageUrlUpload.ts (L23–L24) — sink: fetch(url) with user-controlled URL
```

Evidence

routes/profileImageUrlUpload.ts:23-24:

const url = req.body.imageUrl

const response = await fetch(url)

Suggested fix

Validate URLs against an allowlist of external domains. Block private IP ranges and cloud metadata endpoints.

Finding 11: XML External Entity (XXE) Injection via File Upload Enables File Read and SSRF

ID	**f-09a4adda-f1c1-4602-b63b-bb102043d697**
Severity	**High**
Risk Type	**Xml External Entity**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:18 PM (GMT+7)**

Description

The XML file upload parsing enables external entity loading (XML_PARSE_NOENT | XML_PARSE_DTD-LOAD) and registers filesystem input providers. This allows XXE injection to read arbitrary local files (file:///etc/passwd) and perform SSRF via HTTP entity references (http://169.254.169.254/...).

An attacker can read arbitrary server files (credentials, keys) and perform SSRF to access internal services like AWS metadata.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/lib/xml.ts (L34–L37) — sink: XML parsing with external entity loading enabled
```

Evidence

lib/xml.ts:34-37:

const option = libxml2.ParseOption.XML_PARSE_NOENT | libxml2.ParseOption.XML_PARSE_DTDLOAD | libxml2.ParseOption.XML_PARSE_NOBLANKS | libxml2.ParseOption.XML_PARSE_NOCDATA

// Also: xmlRegisterFsInputProviders() grants WASM sandbox filesystem access

Suggested fix

Disable external entity loading: remove XML_PARSE_DTDLOAD and XML_PARSE_NOENT flags. Do not call xmlRegisterFsInputProviders().

Finding 12: Unsalted MD5 Password Hashing Enables Trivial Hash Reversal

ID	**f-f6e75077-abf9-4529-a786-cffd84c1902e**
Severity	**High**
Risk Type	**Cryptographic Vulnerabilities**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:19 PM (GMT+7)**

Description

Passwords are hashed using MD5 (crypto.createHash('md5')) without salting. MD5 is cryptographically broken and rainbow table attacks can reverse most common passwords instantly.

An attacker who obtains the password hashes (via SQL injection or database access) can trivially reverse them using precomputed rainbow tables.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/lib/insecurity.ts (L41) — sink: unsalted MD5 password hashing
```

Evidence

lib/insecurity.ts:41:

export const hash = (data: string) => crypto.createHash('md5').update(data).digest('hex')

Suggested fix

Use bcrypt, scrypt, or Argon2 with per-user salts for password hashing.

Finding 13: HTTP ALB Listener Forwards Traffic Without TLS Redirect

ID	**f-7f458f66-271a-4106-b8ea-19a96de3d5f3**
Severity	**High**
Risk Type	**Security Misconfiguration**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:27 PM (GMT+7)**

Description

The Terraform ALB configuration has an HTTP listener (port 80) with action type 'forward' instead of 'redirect' to HTTPS. This means all HTTP traffic is served over cleartext even though HTTPS is configured.

An attacker on the network can intercept all HTTP traffic including JWT tokens, passwords in query strings, and payment data.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/ter-raform/networking.tf (L157–L166) — sink: HTTP listener forwards without HTTPS redirect
```

Evidence

terraform/networking.tf:157-166 - HTTP listener uses type=forward instead of redirect to HTTPS

Suggested fix

Change HTTP listener action from 'forward' to 'redirect' with protocol = HTTPS and status_code = HTTP_301.

Finding 14: Password Hash and TOTP Secret Leaked in JWT Payload

ID	**f-6e5b8f0c-3879-4b54-9c76-b2a163009e19**
Severity	**High**
Risk Type	**Information Disclosure**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:26 PM (GMT+7)**

Description

The JWT signing function includes the full user object in the JWT payload, including the password hash and totpSecret. Since JWTs are base64-encoded (not encrypted), any token holder can decode the payload to reveal sensitive data.

An attacker with a valid JWT (or who intercepts one) can decode it to obtain the user's password hash and TOTP secret.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/lib/insecurity.ts (L54) — sink: JWT signing includes full user object with password
hash
```

Evidence

lib/insecurity.ts:54:

export const authorize = (user = {}) => jwt.sign(user, privateKey, { expiresIn: '6h', algorithm: 'RS256' })

// 'user' contains full user object including password hash

Suggested fix

Only include necessary claims in JWT (id, email, role). Never include password hashes or secrets.

Finding 15: Missing Authorization on Product Update Endpoint

ID	**f-c371d151-ea63-4a45-a4bf-6e1966d5b9ea**
Severity	**High**
Risk Type	**Broken Object Property Authorization**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:20 PM (GMT+7)**

Description

The PUT /api/Products/:id endpoint has no authorization middleware, allowing any unauthenticated user to modify product data including descriptions, prices, and names.

An attacker can modify product descriptions to inject XSS payloads (rendered via bypassSecurityTrustHtml), change prices, or deface the application.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/server.ts (L389) — sink: unauthenticated product update endpoint
```

Evidence

server.ts:389 - Product update without authorization middleware (no isAuthorized() call)

Suggested fix

Add isAuthorized() and admin role check middleware to the PUT /api/Products/:id route.

Finding 16: NoSQL Injection in Review Update Enables Mass Modification

ID	**f-208f6765-3b99-4ab4-ba15-2a3edf173546**
Severity	**High**
Risk Type	**Sql Injection**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:18 PM (GMT+7)**

Description

The review update endpoint uses MongoDB's update with `multi: true` and accepts user-controlled `_id` filter. By injecting a NoSQL operator like `{$ne: ''}` in the id field, an attacker can modify all reviews at once.

An attacker can mass-modify all product reviews in the database with a single request.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/routes/updateProductReviews.ts (L16–L19) — sink: MongoDB update with multi:true and user-controlled filter
```

Evidence

routes/updateProductReviews.ts:16-19:

db.reviewsCollection.update(

{ _id: req.body.id },

{ $set: { message: req.body.message } },

{ multi: true }

)

Suggested fix

Remove multi:true, validate _id is a proper ObjectId string, and verify review ownership.

Finding 17: Stored XSS via User Email Registration Targeting Admin Panel via bypassSecurityTrustHtml

ID	**f-49762066-7a99-4e31-b4cb-6d6ffbc35629**
Severity	**High**
Risk Type	**Cross Site Scripting**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:16 PM (GMT+7)**

Description

User email addresses are rendered using `[innerHTML]` binding in the admin panel (frontend/src/app/administration/administration.component.html:26,113) after being processed through `DomSanitizer.bypassSecurityTrustHtml()` at administration.component.ts:74. The `bypassSecurityTrustHtml()` call wraps the email in a `<span>` tag but passes through the raw email value without sanitization: `this.sanitizer.bypassSecurityTrustHtml(\<span class="...">${user.email}</span>\)`. Since user registration (POST /api/Users) does not validate the email format beyond trimming whitespace (server.ts:427-430), an attacker can register with an XSS payload as their email.

An attacker registers with email `<img src=x onerror=alert(document.cookie)>` via POST /api/Users (unauthenticated). When an admin views the administration panel, the malicious email is rendered as HTML via innerHTML, executing JavaScript in the admin's browser context. This enables session theft of the admin JWT token (stored in localStorage and non-httpOnly cookie), granting the attacker full admin access.

Verified: The [innerHTML] binding at administration.component.html:26 renders user.email which was processed through `bypassSecurityTrustHtml()` at line 74 of the component — this explicitly tells Angular to skip XSS sanitization. User registration at server.ts:427-430 only trims the email, does not validate format or sanitize HTML. The finale-rest User model excludes password/totpSecret but not email from the listing endpoint.

Could not verify: Whether Angular's DomSanitizer strips dangerous content despite bypassSecurityTrustHtml (it explicitly does not — that's the purpose of the bypass). Whether CSP prevents inline script execution (no CSP is configured, confirmed at server.ts:186-187).

Code locations

`/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/frontend/src/app/administration/administration.component.ts (L74) — sink: bypassSecurityTrustHtml on user email disables XSS protection`

`/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/frontend/src/app/administration/administration.component.html (L26) — sink: [innerHTML] renders unsafe email HTML`

`/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/server.ts (L427–L430) — source: user registration only trims email, no format validation`

Evidence

frontend/src/app/administration/administration.component.ts:74:

user.email = this.sanitizer.bypassSecurityTrustHtml(`<span class="${...}">${user.email}</span>`)

// bypassSecurityTrustHtml explicitly disables Angular's XSS protection

frontend/src/app/administration/administration.component.html:26:

<mat-cell *matCellDef="let user" [innerHTML]="user.email" class="cell-middle"></mat-cell>

// innerHTML renders the bypassed HTML including any XSS payload

Source - user registration (server.ts:427-430):

if (req.body.email.length !== 0 && req.body.password.length !== 0) {

req.body.email = req.body.email.trim() // Only trims, no format validation

}

// POST /api/Users is unauthenticated, attacker can register with HTML email

GET /api/Users (server.ts:382) requires isAuthorized() but returns all users to any authenticated user

Suggested fix

Remove bypassSecurityTrustHtml and validate email format at registration.

1. At frontend/src/app/administration/administration.component.ts:74, remove the `bypassSecurityTrustHtml()` call and use Angular's default sanitization or text interpolation (`{{ user.email }}`) instead of innerHTML.
2. At server.ts:427-430, add email format validation using a regex or validator library to reject non-email values before they reach the database.
3. If HTML rendering is needed for the active session indicator, construct the HTML with the email as text content (not interpolated into HTML): use `textContent` or Angular's built-in escaping.

Finding 18: Predictable OAuth User Password Derived from Reversed Email Enables Account Takeover

ID	**f-38224644-bdfe-4cfc-9f07-01bff39bbffd**
Severity	**High**
Risk Type	**Authentication Bypass**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:16 PM (GMT+7)**

Description

OAuth users are registered with a password derived from their email address using a trivially reversible algorithm exposed in the client-side JavaScript bundle. At frontend/src/app/oauth/oauth.component.ts:29, the password is computed as `btoa(profile.email.split('').reverse().join(''))` — base64-encode the reversed email. This same computation is used again at line 42 for login. Since this code is part of the Angular frontend bundle delivered to all users' browsers, any attacker can compute any OAuth user's password from their email address alone.

An attacker who knows a user's email (obtainable via GET /api/Users which returns all user emails to any authenticated user, per server.ts:382/497) can compute their password and log in directly via POST /rest/user/login without needing OAuth. This enables account takeover of all OAuth-registered users. For example, for email `bjoern.kimminich@gmail.com`, the password is `btoa('moc.liamg@hcinimmik.nreojb')` = `bW9jLmxpYW1nQGhjaW5pbW1pay5ucmVvamI=`.

Verified: The algorithm is visible at frontend/src/app/oauth/oauth.component.ts:29 (registration) and line 42 (login). This is a deterministic derivation with no server-side random component. The challenge verification in routes/login.ts:73 confirms this exact password for `bjoern.kimminich@gmail.com` as `bW9jLmxpYW1nQGhjaW5pbW1pay5ucmVvamI=`. No additional authentication factor is required for OAuth users (unless they manually set up 2FA).

Could not verify: Whether the production build obfuscates this code enough to prevent discovery (it's in a standard Angular bundle which is easily readable).

Code locations

`/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/frontend/src/app/oauth/oauth.component.ts (L29) — sink: predictable OAuth password derivation exposed in client bundle`

`/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/frontend/src/app/oauth/oauth.component.ts (L42) — evidence: same derivation used for login`

`/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/routes/login.ts (L73) — confirmation: challenge verifies known password for OAuth user`

Evidence

frontend/src/app/oauth/oauth.component.ts:29:

const password: [REDACTED]'').reverse().join(''))

frontend/src/app/oauth/oauth.component.ts:42 (login function):

this.userService.login({ email: profile.email, password: [REDACTED], oauth: true })

Confirmed by routes/login.ts:73:

challengeUtils.solveIf(challenges.oauthUserPasswordChallenge, () => {

return req.body.email === 'bjoern.kimminich@gmail.com' && req.body.password: [REDACTED] 'bW9jLmxpYW1nQGhjaW5pbW1pay5ucmVvamI='

})

// 'bW9jLmxpYW1nQGhjaW5pbW1pay5ucmVvamI=' = btoa('moc.liamg@hcinimmik.nreojb') = btoa(reverse('bjoern.kimminich@gmail.com'))

Suggested fix

Generate cryptographically random passwords for OAuth users and do not expose the derivation logic in client code.

1. At frontend/src/app/oauth/oauth.component.ts:29, replace the deterministic password with a cryptographically random value: use a secure random generator (e.g., `crypto.getRandomValues()`) to create a strong random password for each OAuth registration.
2. Alternatively, implement proper OAuth token-based authentication where OAuth users are authenticated solely through their OAuth provider token, without a stored password in the application database.
3. Reset all existing OAuth user passwords to random values and invalidate their sessions to prevent exploitation of already-computed passwords.

Finding 19: Unrestricted CORS Policy Allows Cross-Origin Data Theft

ID	**f-cfa46a51-15d4-42d2-8cdc-c36c499758cd**
Severity	**High**
Risk Type	**Security Misconfiguration**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:19 PM (GMT+7)**

Description

The application uses unrestricted CORS configuration (cors() without origin specification), allowing any origin to make authenticated cross-origin requests.

Combined with JSONP endpoints and sensitive data exposure, this enables cross-origin data theft from any malicious website.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/server.ts (L182–L183) — sink: unrestricted CORS configuration
```

Evidence

server.ts:182-183:

app.options('*', cors())

app.use(cors())

Suggested fix

Configure CORS with specific allowed origins: cors({ origin: 'https://yourdomain.com', credentials: true })

Finding 20: Null Byte Injection Bypasses File Extension Allowlist in FTP Server

ID	**f-a45642eb-ccdf-4cea-9358-526d711b37dc**
Severity	**High**
Risk Type	**Path Traversal**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:20 PM (GMT+7)**

Description

The FTP file server checks the file extension before applying the null byte cutoff, allowing bypass of the .md/.pdf allowlist. A request for `file.md%00.bak` passes the extension check (ends with allowlisted type after null byte) but the null byte is cut off afterward, serving the actual file.

An attacker can access any file in the ftp/ directory regardless of extension restriction by appending `%00.md` or `%00.pdf`.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/routes/fileServer.ts (L26–L32) — sink: null byte cutoff after extension check allows
arbitrary file read
```

Evidence

routes/fileServer.ts:26-30:

if (file && (endsWithAllowlistedFileType(file) || (file === 'incident-support.kdbx'))) {

 file = security.cutOffPoisonNullByte(file) // Null byte removed AFTER extension check

 res.sendFile(path.resolve('ftp/', file))

}

Suggested fix

Apply null byte sanitization BEFORE the extension check, or reject any filename containing null bytes.

Finding 21: Password Hash Leak and JSONP Cross-Origin Data Theft via whoami Endpoint

ID	**f-def1d9df-a3aa-4629-aaa7-d140817f8282**
Severity	**High**
Risk Type	**Information Disclosure**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:20 PM (GMT+7)**

Description

The whoami endpoint (routes/currentUser.ts) allows arbitrary field selection via a `fields` query parameter, exposing sensitive fields like password hash and totpSecret. Additionally, the endpoint supports JSONP via `req.query.callback`, enabling cross-origin data theft.

An attacker can extract password hashes and TOTP secrets for any authenticated user, and combine with JSONP to exfiltrate data cross-origin.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/routes/currentUser.ts (L22–L32) — sink: arbitrary field selection exposes password hash
```

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/routes/currentUser.ts (L46) — sink: JSONP callback enables cross-origin data exfiltration
```

Evidence

routes/currentUser.ts:22-32:

const fieldsParam = req.query?.fields as string | undefined

const requestedFields = fieldsParam ? fieldsParam.split(',').map(f => f.trim()) : []

// When fields are specified, return only those fields (including password!)

for (const field of requestedFields) {

if (user?.data[field] !== undefined) { baseUser[field] = user?.data[field] }

}

// Line 46: res.jsonp(response) -- JSONP callback enabled

Suggested fix

Remove the fields parameter entirely. Never expose password hashes via API. Remove JSONP support.

Finding 22: Unauthenticated Access to Server Log Files Containing Sensitive Data

ID	**f-6672b79c-8128-4c50-88b9-4bcb9cbfe287**
Severity	**High**
Risk Type	**Information Disclosure**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:25 PM (GMT+7)**

Description

Server log files are served without any authentication at /support/logs. The logs contain access log data including full URLs with passwords (from GET-based password change), IP addresses, and other sensitive information.

An unauthenticated attacker can read all server logs, potentially harvesting user passwords and session information.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/server.ts (L300–L302) — sink: unauthenticated log file serving
```

Evidence

server.ts:300-302:

app.use('/support/logs', serveIndexMiddleware, serveIndex('logs', {...}))

app.use('/support/logs/:file', serveLogFiles())

Suggested fix

Add authentication and admin role check to the log serving endpoints. Redact sensitive data from logs.

Finding 23: Wallet Balance Manipulation via Missing Positive-Value Validation

ID	**f-5c2265d8-8927-4001-aa82-58b9d295dde8**
Severity	**High**
Risk Type	**Business Logic Vulnerabilities**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:21 PM (GMT+7)**

Description

The wallet balance increment endpoint does not validate that the balance value is positive. An attacker can send a negative balance to effectively withdraw money, or manipulate balances without proper financial controls.

An attacker can inflate their wallet balance by sending negative values or bypass payment requirements.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/routes/wallet.ts (L26) — sink: wallet balance increment without positive-value
validation
```

Evidence

routes/wallet.ts:26:

await WalletModel.increment({ balance: req.body.balance }, { where: { UserId: req.body.UserId } })

// No check that req.body.balance > 0

Suggested fix

Validate that balance > 0 before incrementing. Implement server-side validation for all financial operations.

Finding 24: Zip Slip Path Traversal - Arbitrary File Write (Unauthenticated)

ID	**f-14ecf89d-126f-47d3-a8c1-af9432be2587**
Severity	**High**
Risk Type	**Path Traversal**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:24 PM (GMT+7)**

Description

ZIP file extraction constructs output paths by concatenating the entry filename without proper validation. While there's a check that `absolutePath.includes(path.resolve('.'))`, a crafted zip with `../` entries can write outside the intended directory.

An attacker can overwrite arbitrary files on the server via Zip Slip (path traversal in ZIP entries).

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/routes/fileUpload.ts (L30–L35) — sink: ZIP extraction without proper path traversal validation
```

Evidence

routes/fileUpload.ts:30-35:

const fileName = entry.path

const absolutePath = path.resolve('uploads/complaints/' + fileName)

if (absolutePath.includes(path.resolve('.'))) {

await pipeline(entry.stream(), fs.createWriteStream('uploads/complaints/' + fileName))

}

Suggested fix

Validate that extracted file paths don't contain '..' and that the resolved path is within the intended directory.

Finding 25: CSRF Password Change via GET Request Without Current Password Verification

ID	**f-4c08a8e4-cacf-498e-9c2a-b392fe204f67**
Severity	**High**
Risk Type	**Authentication Bypass**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:20 PM (GMT+7)**

Description

The password change is implemented as a GET request with passwords in query parameters. Additionally, the `current` password parameter is optional - if not provided, the password is changed without verifying the existing password.

An attacker can change any authenticated user's password via CSRF (GET request), and the current password verification is completely optional.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/routes/changePassword.ts (L13–L56) — sink: GET-based password change with optional
current password verification
```

Evidence

routes/changePassword.ts:13-14,44:

const currentPassword: [REDACTED] as string

const newPassword: [REDACTED] as string

// Line 44: if (currentPassword && ...) -- current password check is conditional!

Suggested fix

Use POST method for state-changing operations. Always require current password. Implement CSRF tokens.

Finding 26: SQL Injection in Product Search via Query Parameter

ID	**f-ad6a856b-ef62-455b-9b4d-df4cf16fa8b6**
Severity	**High**
Risk Type	**Sql Injection**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:17 PM (GMT+7)**

Description

The product search endpoint constructs a raw SQL query by directly interpolating user-supplied search criteria into the query string without parameterization.

An attacker can extract all database contents (user credentials, credit cards) via UNION-based SQL injection.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/routes/search.ts (L22) — sink: raw SQL query with string interpolation of search criteria
```

Evidence

routes/search.ts:22:

models.sequelize.query(`SELECT * FROM Products WHERE ((name LIKE '%${criteria}%' OR description LIKE '%${criteria}%') AND deletedAt IS NULL) ORDER BY name`)

Suggested fix

Use parameterized queries with LIKE wildcards as bound parameters.

Finding 27: Stored XSS via Unauthenticated Product Description Modification

ID	**f-c894f38f-7ba4-4987-8ba4-50ae1192fbe6**
Severity	**High**
Risk Type	**Cross Site Scripting**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:24 PM (GMT+7)**

Description

Product descriptions can be modified by any unauthenticated user via PUT /api/Products/:id (no auth middleware). The Angular frontend renders product descriptions using bypassSecurityTrustHtml, enabling stored XSS affecting all users.

An attacker can inject persistent JavaScript payloads visible to every user viewing the product page, enabling session theft and account takeover.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/server.ts (L389) — source: unauthenticated PUT /api/Products/:id
```

Evidence

server.ts:389 - PUT /api/Products/:id has no auth

Angular frontend renders descriptions via bypassSecurityTrustHtml

Suggested fix

Add authentication and admin authorization to product modification endpoints. Sanitize product descriptions or remove bypassSecurityTrustHtml.

Finding 28: Unauthenticated Access to JWT Public Key Enabling Algorithm Confusion Attack

ID	**f-5e66f99f-7dde-43e0-a963-c43fb3a39f59**
Severity	**High**
Risk Type	**Information Disclosure**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:25 PM (GMT+7)**

Description

The /encryptionkeys directory is served without authentication, exposing the JWT public key file (jwt.pub). Combined with the algorithm confusion vulnerability (T12), an attacker can use this public key to forge valid JWT tokens with HS256 algorithm.

An attacker can obtain the public key and use it with HS256 algorithm to sign forged JWT tokens that pass verification.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/server.ts (L296–L297) — sink: unauthenticated encryption key directory listing and access
```

Evidence

server.ts:296-297:

app.use('/encryptionkeys', serveIndexMiddleware, serveIndex('encryptionkeys', {...}))

app.use('/encryptionkeys/:file', serveKeyFiles())

Suggested fix

Remove the /encryptionkeys directory listing. Even though public keys are theoretically public, serving them alongside an algorithm confusion vulnerability enables token forgery.

Finding 29: 2FA Bypass via Forged tmpToken and Rate Limit Bypass

ID	**f-b6b0f188-9cce-4375-88fa-b29503a8d089**
Severity	**High**
Risk Type	**Authentication Bypass**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:26 PM (GMT+7)**

Description

The 2FA verification uses a tmpToken that can be forged using the hardcoded HMAC key ('pa4qacea4VK9t9nGv7yZtwmj'). Additionally, the rate limiting on 2FA verification can be bypassed via X-Forwarded-For spoofing (trust proxy enabled), enabling brute force of the 6-digit TOTP code.

An attacker can forge tmpTokens to bypass the password verification step, or brute-force the 6-digit TOTP code within its 30-second validity window.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/routes/2fa.ts (L16–L45) — sink: 2FA verification with forgeable tmpToken and rate
limit bypass
```

Evidence

routes/2fa.ts:16-45 - 2FA verification with forgeable tmpToken and bypassable rate limit

Suggested fix

Use secure random tokens stored server-side instead of HMAC-based tokens. Use non-IP-based rate limiting.

Finding 30: Credit Card Numbers Stored in Plaintext Without Encryption

ID	**f-934f508d-081e-4c4b-9be2-99dd0c39fba3**
Severity	**High**
Risk Type	**Cryptographic Vulnerabilities**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:24 PM (GMT+7)**

Description

Credit card numbers are stored as plaintext integers in the database without any encryption or tokenization. The model defines cardNum as a plain integer with validation only checking min/max range.

If the database is compromised via SQL injection or direct access, all credit card numbers are immediately exposed in cleartext.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/models/card.ts (L38–L44) — sink: credit card numbers stored as plaintext integers
```

Evidence

models/card.ts:38-44:

cardNum: { type: DataTypes.INTEGER, validate: { min: 1000000000000000, max: 9999999999999998 } }

Suggested fix

Use a payment tokenization service. Never store full card numbers. If needed, encrypt at rest with a separate key management system.

Finding 31: Default Admin Credentials Enable Unauthenticated Administrative Access

ID	**f-291c6b38-572c-4cf6-b0b7-b06d54f2b5f4**
Severity	**High**
Risk Type	**Default Credentials**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:20 PM (GMT+7)**

Description

The application seeds a default admin user with the weak password 'admin123'. This password is easily guessable and provides full administrative access.

An attacker can log in as admin with trivially guessable credentials (admin@juice-sh.op / admin123).

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/server.ts (L427) — config: application seeds default admin with weak password
```

Evidence

Default admin user seeded with password 'admin123' in data/static/users.yml

Suggested fix

Force password change on first login. Use strong randomly-generated initial passwords. Implement password complexity requirements.

Finding 32: Unauthenticated Exposure of Security Question Answers Enabling Account Takeover

ID	**f-69891b64-75af-4b6c-9696-3b73f8c790d5**
Severity	**High**
Risk Type	**Information Disclosure**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:27 PM (GMT+7)**

Description

The unauthenticated application configuration endpoint exposes security question answers for seeded users. These answers are used for password reset, enabling account takeover.

An attacker can retrieve security answers from the configuration endpoint and use them to reset passwords for all seeded user accounts.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/routes/appConfiguration.ts (L10) — sink: unauthenticated config exposure includes security answers
```

Evidence

GET /rest/admin/application-configuration exposes security answers in config; routes/resetPassword.ts uses these answers for password reset

Suggested fix

Remove security answers from configuration response. Add authentication to admin endpoints.

Finding 33: IDOR - Checkout Any User's Basket Without Ownership Verification

ID	**f-2c6d0ba9-e781-41fc-a7d7-596211601311**
Severity	**High**
Risk Type	**Insecure Direct Object Reference**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:23 PM (GMT+7)**

Description

The checkout/order placement endpoint accepts a basket ID from the URL parameter without verifying that the basket belongs to the authenticated user. An attacker can checkout another user's basket.

An attacker can place orders using other users' baskets, potentially causing unauthorized purchases or accessing basket contents.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/routes/order.ts (L37) — sink: checkout without basket ownership verification
```

Evidence

routes/order.ts:37:

const id = req.params.id

BasketModel.findOne({ where: { id }, ... })

// No check that basket belongs to the authenticated user

Suggested fix

Verify that the basket's UserId matches the authenticated user before processing checkout.

Finding 34: Local File Inclusion via Layout Parameter in Data Erasure Endpoint

ID	**f-cc02f57a-af22-431c-ad1c-03ab80b0e439**
Severity	**High**
Risk Type	**Local File Inclusion**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:19 PM (GMT+7)**

Description

The data erasure endpoint accepts a `layout` parameter in the request body that is used as a file path for Pug template rendering. While some paths are blocked (ftp, ctf.key, encryptionkeys), an attacker can traverse to other files on the filesystem.

An attacker can read arbitrary files from the server by specifying path traversal sequences in the layout parameter, as long as the file path doesn't contain the blocked keywords.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/routes/dataErasure.ts (L103–L115) — sink: path.resolve(req.body.layout) used in
template rendering
```

Evidence

routes/dataErasure.ts:103-104:

const filePath: string = path.resolve(req.body.layout).toLowerCase()

const isForbiddenFile: boolean = (filePath.includes('ftp') || filePath.includes('ctf.key') || filePath.includes('en-cryptionkeys'))

// If not forbidden, renders the file as a Pug template

Suggested fix

Remove the layout parameter from user input. Use a fixed template for data erasure results.

Finding 35: Hardcoded Ethereum Mnemonic Phrase in Source Code

ID	**f-00344ce0-2d48-43a5-8f4a-2354d0e175e0**
Severity	**High**
Risk Type	**Information Disclosure**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:28 PM (GMT+7)**

Description

An Ethereum wallet mnemonic phrase is hardcoded in the source code, giving anyone with source code access full control over the associated cryptocurrency wallet.

An attacker can derive the private keys from the mnemonic and drain any cryptocurrency held in the associated wallet.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/routes/checkKeys.ts (L10) — sink: hardcoded Ethereum mnemonic phrase
```

Evidence

routes/checkKeys.ts:10:

const mnemonic = 'purpose betray marriage blame crunch monitor spin slide donate sport lift clutch'

Suggested fix

Store mnemonic in environment variables or a secrets management service. Never commit crypto keys to source control.

Finding 36: Rate Limit Bypass via X-Forwarded-For Header Spoofing

ID	**f-90e07448-3a57-40ae-bbf6-9a69ec83abdf**
Severity	**Medium**
Risk Type	**Authentication Bypass**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:19 PM (GMT+7)**

Description

The application enables `trust proxy` (server.ts:361) which causes Express to trust X-Forwarded-For headers for IP determination. Rate limiting is keyed on client IP, so an attacker can bypass rate limits by rotating the X-Forwarded-For header value.

An attacker can bypass all rate limits (login brute force protection, etc.) by setting different X-Forwarded-For values on each request.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/server.ts (L361) — sink: trust proxy enables X-Forwarded-For spoofing for rate limit bypass
```

Evidence

server.ts:361:

app.enable('trust proxy')

Suggested fix

Configure trust proxy to only trust known proxy IPs, or use a rate limiting strategy that doesn't solely rely on IP.

Finding 37: IDOR in PUT /api/Addresss/:id Allows Modifying Any User's Address

ID	**f-a5abf9de-def0-416d-8783-088f6660bfe8**
Severity	**Medium**
Risk Type	**Insecure Direct Object Reference**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:28 PM (GMT+7)**

Description

The address update endpoint (PUT /api/Addresss/:id) lacks ownership verification, allowing any authenticated user to modify any other user's address.

An attacker can modify shipping addresses of other users, potentially redirecting deliveries or accessing personal address information.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/routes/address.ts (L14–L20) — sink: address update without ownership verification
```

Evidence

PUT /api/Addresss/:id - address update lacks ownership verification

Suggested fix

Add ownership check: verify that the address belongs to the authenticated user before allowing modification.

Finding 38: Coupon Forgery via Trivial z85 Encoding Without Cryptographic Signature

ID	**f-4dd40076-8c1e-42cc-ac1c-4dd33794c5e9**
Severity	**Medium**
Risk Type	**Cryptographic Vulnerabilities**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:22 PM (GMT+7)**

Description

Coupons are z85-encoded strings with format 'MMMYY-discount' (e.g., 'JAN25-20'). There is no cryptographic signature - any user who knows the format can forge valid coupon codes for arbitrary discounts.

An attacker can generate valid coupon codes for any discount percentage by simply z85-encoding the current month/year with a desired discount.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/routes/coupon.ts (L10–L20) — sink: coupon verification uses only z85 decoding without cryptographic signature
```

Evidence

lib/insecurity.ts:97-107:

export const generateCoupon = (discount: number, date = new Date()) => {

 const coupon = utils.toMMMYY(date) + '-' + discount

 return z85.encode(coupon)

}

// Verification: z85.decode(coupon) then check format matches MMMYY-NN

Suggested fix

Add HMAC signature to coupons. Validate coupon codes against a server-side database of issued coupons.

Finding 39: IDOR - Any Authenticated User Can Access Any Basket

ID	**f-3db91ded-f79d-4b22-9688-cb4a4cc73913**
Severity	**Medium**
Risk Type	**Insecure Direct Object Reference**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:21 PM (GMT+7)**

Description

The basket retrieval endpoint checks authentication but does not verify that the requested basket belongs to the authenticated user. Any authenticated user can access any other user's basket by changing the ID parameter.

An attacker can view other users' shopping baskets, revealing their items, quantities, and potentially personal preferences.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/routes/basket.ts (L16–L23) — sink: basket retrieval without ownership verification
```

Evidence

routes/basket.ts:16-23:

const id = req.params.id

const basket = await BasketModel.findOne({ where: { id }, include: [...] })

// No check that basket.UserId === authenticated user's ID

Suggested fix

Add ownership check: verify that basket.UserId matches the authenticated user's ID before returning data.

Finding 40: Forged Feedback UserId Allows Impersonation

ID	**f-f9069041-5146-4003-845f-047ce3fbc547**
Severity	**Medium**
Risk Type	**Insecure Direct Object Reference**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:26 PM (GMT+7)**

Description

The feedback submission endpoint (POST /api/Feedbacks) accepts a UserId field in the request body without verifying it matches the authenticated user. Any user can submit feedback impersonating another user.

An attacker can submit feedback attributed to other users, including admin or high-profile accounts.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/server.ts (L420) — sink: feedback UserId not verified against authenticated user
```

Evidence

server.ts:420 - POST /api/Feedbacks accepts body.UserId without verification

Suggested fix

Derive UserId from the authenticated user's JWT token using appendUserId middleware.

Finding 41: Open Redirect via Substring-Based Allowlist Bypass

ID	**f-61bf0edb-bc0f-4147-8b35-f3c7cb0bcc95**
Severity	**Medium**
Risk Type	**Cross Site Scripting**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:21 PM (GMT+7)**

Description

The redirect endpoint uses a substring-based allowlist check (`url.includes(allowedUrl)`) which can be bypassed by including an allowed URL as a substring in a malicious URL.

An attacker can redirect users to malicious sites for phishing by crafting URLs that contain an allowed domain as a substring.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/routes/redirect.ts (L14) — sink: redirect with substring-based allowlist bypass
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/lib/insecurity.ts (L131) — source: substring-based URL allowlist check
```

Evidence

lib/insecurity.ts:131:

allowed = allowed || url.includes(allowedUrl)

// Attack: /redirect?to=https://evil.com?blockchain.info redirects to evil.com

Suggested fix

Use strict URL comparison with parsed URL objects. Check that the URL starts with allowed origins rather than just containing them.

Finding 42: Captcha Answer Leaked in API Response Enables Automated Bypass

ID	**f-360b2c6f-b298-4081-838a-66cb1bd582f9**
Severity	**Medium**
Risk Type	**Authentication Bypass**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:26 PM (GMT+7)**

Description

The CAPTCHA endpoint returns the answer in the API response alongside the challenge, allowing automated tools to solve CAPTCHAs programmatically.

An attacker can bypass CAPTCHA protections by reading the answer from the API response before submitting it.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/routes/captcha.ts (L12–L20) — sink: captcha answer leaked in API response
```

Evidence

routes/captcha.ts:12-20 - GET /rest/captcha returns captcha answer in response

Suggested fix

Never include the answer in the response. Store answers server-side and validate on submission.

Finding 43: LLM Prompt Injection Bypasses Coupon Policy for Arbitrary Discount Generation

ID	**f-b41b9e4b-d9c7-4e4a-80e2-5977f2e95086**
Severity	**Medium**
Risk Type	**Privilege Escalation**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:26 PM (GMT+7)**

Description

The chatbot LLM has a coupon generation tool. The coupon policy is enforced only via the system prompt, which can be bypassed through prompt injection techniques to generate coupons with arbitrary discount percentages.

An attacker can use prompt injection to override the system prompt constraints and generate high-value discount coupons.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/routes/chat.ts (L99) — sink: LLM chatbot with coupon generation tool vulnerable to prompt injection
```

Evidence

routes/chat.ts:99 - LLM chatbot with generateCoupon tool, policy enforced only by system prompt

Suggested fix

Enforce discount limits server-side (not in prompt). Validate coupon values before applying. Use a separate validation layer independent of LLM output.

Finding 44: Stored XSS in Feedback via sanitize-html v1.4.2 Bypass

ID	**f-98b6410a-9b1e-492c-81e9-60021ba89cc6**
Severity	**Medium**
Risk Type	**Cross Site Scripting**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:27 PM (GMT+7)**

Description

The application uses sanitize-html library for HTML sanitization of feedback comments, but with a version (v1.4.2) that has known bypasses. Attackers can craft HTML that evades the sanitizer and executes JavaScript.

An attacker can submit feedback with XSS payloads that bypass the outdated sanitizer, achieving stored XSS affecting all users who view the feedback.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/lib/insecurity.ts (L56) — sink: sanitize-html v1.4.2 with known bypass vulnerabilities
```

Evidence

lib/insecurity.ts:56:

export const sanitizeHtml = (html: string) => sanitizeHtmlLib(html)

// sanitize-html v1.4.2 has known bypass CVEs

Suggested fix

Update sanitize-html to the latest version. Apply recursive sanitization.

Finding 45: Review Author Spoofing via Unverified Author Field

ID	**f-612fae33-e8a7-4b81-9422-b8807d2f216d**
Severity	**Medium**
Risk Type	**Broken Object Property Authorization**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:22 PM (GMT+7)**

Description

The create product review endpoint accepts an `author` field from the request body without verifying it matches the authenticated user's email. Any user can submit reviews as any other user.

An attacker can post defamatory or fraudulent reviews attributed to other users.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/routes/createProductReviews.ts (L22) — sink: review creation with unverified author field
```

Evidence

routes/createProductReviews.ts:22:

// body.author accepted without check against authenticated user

Suggested fix

Derive the author from the authenticated user's JWT token, not from request body.

Finding 46: Missing Authorization Check on Review Update (Any User Can Edit Any Review)

ID	**f-afc1c410-e82c-40e3-b06c-e45426973170**
Severity	**Medium**
Risk Type	**Insecure Direct Object Reference**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:22 PM (GMT+7)**

Description

The review update endpoint does not verify that the authenticated user is the author of the review being updated. Any authenticated user can edit any review.

An attacker can modify reviews written by other users, altering ratings and content.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/routes/updateProductReviews.ts (L16–L19) — sink: review update without ownership verification
```

Evidence

routes/updateProductReviews.ts:16:

db.reviewsCollection.update({ _id: req.body.id }, { $set: { message: req.body.message } }, ...)

// No ownership check

Suggested fix

Add ownership check: verify that the review's author matches the authenticated user's email before allowing update.

Finding 47: Hardcoded Test Credentials Exposed in Angular Frontend Bundle

ID	**f-15d4cbc7-f44e-4a42-b5d7-79a5a17ce166**
Severity	**Medium**
Risk Type	**Default Credentials**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:29 PM (GMT+7)**

Description

Test credentials are hardcoded in the Angular frontend bundle, visible to anyone who inspects the client-side JavaScript code.

An attacker can extract test credentials from the client bundle and use them to log in to a test account.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/frontend/src/app/login/login.component.ts (L76–L77) — sink: hardcoded test
credentials in client bundle
```

Evidence

frontend/src/app/login/login.component.ts:76-77:

public testingUsername = 'testing@juice-sh.op'

public testingPassword: [REDACTED]'

Suggested fix

Remove test credentials from source code. Use environment-specific test accounts that are not deployed to production.

Finding 48: JWT Token Stored in localStorage and Insecure Cookie Enabling Session Theft via XSS

ID	**f-6530932e-3f54-4667-81bb-6bc2b3a12011**
Severity	**Medium**
Risk Type	**Session Token Vulnerabilities**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:17 PM (GMT+7)**

Description

JWT tokens are stored in localStorage (accessible to any JavaScript on the page) in the Angular frontend (frontend/src/app/oauth/oauth.component.ts:51: `localStorage.setItem('token', authentication.token)`) and set as a cookie without httpOnly, secure, or sameSite attributes (lib/insecurity.ts:192: `res.cookie('token', token)` with no options, and server-side updateAuthenticatedUsers at lib/insecurity.ts:192 also sets the cookie without flags). The combination means any XSS vulnerability — including the confirmed stored XSS via email registration — enables complete session theft.

Any XSS payload can read `localStorage.getItem('token')` or `document.cookie` to steal the JWT token. Given the confirmed stored XSS vector (T102) in the admin panel and the lack of Content-Security-Policy (no CSP configured), token theft is straightforward. The stolen token provides full authenticated access for 6 hours with no revocation mechanism.

Verified: lib/insecurity.ts:192 confirms `res.cookie('token', token)` with no options object (no httpOnly, secure, sameSite). Frontend code at oauth.component.ts:51 confirms `localStorage.setItem('token', ...)`. No Content-Security-Policy is configured (server.ts:186-187 only uses noSniff and frameguard). The token cookie is readable by JavaScript.

Could not verify: Whether a CDN or proxy adds cookie security flags. None found in infrastructure code.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/lib/insecurity.ts (L192) — sink: cookie set without httpOnly/secure/sameSite flags
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/frontend/src/app/oauth/oauth.component.ts (L51) — sink: JWT stored in localStorage accessible to XSS
```

Evidence

lib/insecurity.ts:192 (cookie without security flags):

res.cookie('token', token)

// No { httpOnly: true, secure: true, sameSite: 'strict' }

frontend/src/app/oauth/oauth.component.ts:51:

localStorage.setItem('token', authentication.token)

// Accessible to any JS on the page

Combined with XSS vector:

’Stored XSS via email registration (administration.component.ts:74)

’XSS payload: document.cookie reads token, localStorage.getItem('token') reads token

’Token valid for 6 hours, no revocation (lib/insecurity.ts:52, :70-91)

Suggested fix

Set httpOnly, secure, and sameSite flags on the token cookie and avoid localStorage for sensitive tokens.

1. At lib/insecurity.ts:192, change to: `res.cookie('token', token, { httpOnly: true, secure: true, sameSite: 'strict' })`.
2. In the frontend (oauth.component.ts and login.component.ts), remove `localStorage.setItem('token', ...)` and rely solely on the httpOnly cookie for authentication.
3. Adjust the backend to read the token from the httpOnly cookie rather than requiring the Authorization header for browser-based requests.
4. Add a Content-Security-Policy header to prevent inline script execution as defense-in-depth.

Finding 49: Reflected XSS via Track Order Response Rendered with bypassSecurityTrustHtml

ID	**f-88442963-ae2a-479e-ac68-f6e408281b75**
Severity	**Medium**
Risk Type	**Cross Site Scripting**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:22 PM (GMT+7)**

Description

The track order endpoint echoes the orderId parameter in the response (with only truncation to 60 chars). The Angular frontend renders this value using bypassSecurityTrustHtml, enabling reflected XSS.

An attacker can inject JavaScript payloads into the order tracking response that execute in the victim's browser.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/routes/trackOrder.ts (L14) — source: orderId echoed unsanitized in response
```

Evidence

routes/trackOrder.ts:14:

const id = utils.trunc(req.params.id, 60)

// id is echoed directly in the response: result.data[0] = { orderId: id }

Suggested fix

HTML-encode the orderId before including it in the response. Remove bypassSecurityTrustHtml usage in the frontend.

Finding 50: Unauthenticated Order Tracking Reveals Sensitive Order Details

ID	**f-494dce43-c763-4279-af6d-44567b3ca474**
Severity	**Medium**
Risk Type	**Information Disclosure**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:23 PM (GMT+7)**

Description

The order tracking endpoint requires no authentication and returns full order details for any valid order ID. There is no ownership verification.

An attacker can enumerate order IDs and view other users' order details including items, quantities, and prices.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/routes/trackOrder.ts (L12–L14) — sink: unauthenticated order tracking
```

Evidence

routes/trackOrder.ts:12-14 - Unauthenticated endpoint returns order details without ownership check

Suggested fix

Require authentication and verify order belongs to the authenticated user.

Finding 51: Unauthenticated Full Application Configuration Exposure

ID	**f-2af163d7-dc65-4a18-afbb-e556d09a7721**
Severity	**Medium**
Risk Type	**Information Disclosure**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:26 PM (GMT+7)**

Description

The application configuration endpoint (GET /rest/admin/application-configuration) is served without any authentication despite having 'admin' in the URL path. It exposes the full application configuration including chatbot settings, challenge details, and internal settings.

An unauthenticated attacker can access the complete application configuration revealing internal business logic and secrets.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/server.ts (L626) — sink: unauthenticated application configuration exposure
```

Evidence

server.ts:626:

app.get('/rest/admin/application-configuration', retrieveAppConfiguration())

// No auth middleware

Suggested fix

Add authentication and admin role verification middleware to this endpoint.

Finding 52: Authenticated User Enumeration Exposes All User Emails, Roles, and IPs Without Admin Restriction

ID	**f-01851c2d-ee87-465a-97c9-a56286094ea3**
Severity	**Medium**
Risk Type	**Information Disclosure**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:17 PM (GMT+7)**

Description

GET /api/Users (server.ts:382) requires only `isAuthorized()` (valid JWT) and returns all users in the system via the finale-rest auto-generated endpoint. While the `excludeAttributes: ['password', 'totpSecret']` configuration (server.ts:500) prevents password hash exposure through this endpoint, the response still includes: user ID, email, role, profileImage, createdAt, updatedAt, and lastLoginIp for every user in the system. Any authenticated user (including newly registered accounts) can enumerate all users.

An attacker who registers a free account gains access to all user email addresses, roles (identifying admins), and last login IPs. This information enables: (1) targeted attacks against admin accounts, (2) OAuth password computation (given the predictable derivation), (3) social engineering, and (4) correlation of user activity via IP addresses.

Verified: server.ts:382 `app.get('/api/Users', security.isAuthorized())` — only checks for valid JWT, no admin role requirement. The finale-rest configuration at server.ts:500 excludes password and totpSecret but exposes all other User model fields. User registration (POST /api/Users) is unauthenticated, so anyone can create an account to access the user list.

Could not verify: Whether the finale-rest response actually includes lastLoginIp (it depends on the User model fields). The User model includes email, role, profileImage, lastLoginIp based on the profile references.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/server.ts (L382) — sink: user listing endpoint returns all users to any authenticated
user
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/server.ts (L500) — config: finale-rest User resource excludes only password/totpSecret
```

Evidence

server.ts:382:

app.get('/api/Users', security.isAuthorized())

// Only requires valid JWT - any authenticated user, no admin check

server.ts:497-500 (finale-rest auto-generated):

{ name: 'User', exclude: ['password', 'totpSecret'], model: UserModel }

// Excludes password/totpSecret but exposes email, role, profileImage, lastLoginIp, etc.

Attack chain:

1. POST /api/Users (unauthenticated) — register free account

2. POST /rest/user/login — get JWT token

3. GET /api/Users with Authorization: Bearer <token> — get all users

Suggested fix

Restrict the user listing endpoint to admin role and limit the fields returned.

1. At server.ts:382, change to require admin authorization: add a custom middleware that checks `role === 'admin'` after isAuthorized().
2. In the finale-rest User configuration (server.ts:500), add more fields to exclude: `exclude: ['password', 'totpSecret', 'lastLoginIp', 'role']` for non-admin access.
3. If regular users need to look up other users (e.g., for reviews), provide a separate endpoint that returns only minimal public profile data (username, profileImage).

Finding 53: Verbose Error Handler Exposes Stack Traces in Production

ID	**f-66654f0e-64f8-4c73-9c05-cb820d2ae6d0**
Severity	**Medium**
Risk Type	**Information Disclosure**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:20 PM (GMT+7)**

Description

The error handler (errorhandler()) is used without environment check, exposing full stack traces in all environments including production. This reveals internal file paths, framework versions, and code structure.

An attacker can use stack traces to understand the application internals and plan targeted attacks.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/server.ts (L701) — sink: errorhandler() active in all environments
```

Evidence

server.ts:701:

app.use(errorhandler())

// No environment check - always active

Suggested fix

Only use errorhandler() in development: if (process.env.NODE_ENV === 'development') { app.use(errorhandler()) }

Finding 54: LLM Tool Call Arguments Including Coupon Codes Leaked via SSE Stream

ID	**f-3531f3a8-2646-482c-86b7-454b560c2883**
Severity	**Medium**
Risk Type	**Information Disclosure**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:27 PM (GMT+7)**

Description

The chatbot's SSE (Server-Sent Events) stream leaks LLM tool call arguments to the client, including generated coupon codes. This exposes internal tool operations that should be hidden from users.

An attacker can intercept the SSE stream to observe coupon codes and other tool call arguments before they are officially delivered.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/routes/chat.ts (L180) — sink: LLM tool call arguments leaked to client via SSE
```

Evidence

routes/chat.ts:180 - SSE stream exposes tool_calls including coupon codes

Suggested fix

Filter tool_calls from the SSE stream sent to clients. Only send the final text response.

Finding 55: Stored XSS via True-Client-IP Header in Login IP Storage

ID	**f-c0f73275-fc74-422c-a973-9587900aa2f4**
Severity	**Medium**
Risk Type	**Cross Site Scripting**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:21 PM (GMT+7)**

Description

The True-Client-IP header is stored directly in the database without sanitization (when the XSS challenge is enabled). This stored value is later rendered in the admin panel, enabling stored XSS targeting administrators.

An attacker can inject XSS payloads via the True-Client-IP header that execute when an admin views user details.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/routes/saveLoginIp.ts (L19–L24) — sink: True-Client-IP header stored unsanitized
```

Evidence

routes/saveLoginIp.ts:19-21:

let lastLoginIp = req.headers['true-client-ip']

if (utils.isChallengeEnabled(challenges.httpHeaderXssChallenge)) {

// Header stored unsanitized when challenge is enabled

}

Suggested fix

Always sanitize the True-Client-IP header before storing. Use HTML entity encoding when rendering in admin views.

Finding 56: CSP Header Injection via User-Controlled profileImage Field

ID	**f-e07f7039-97b9-438f-a49f-029e3b04dc13**
Severity	**Medium**
Risk Type	**Cross Site Scripting**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:24 PM (GMT+7)**

Description

The user profile rendering constructs a CSP header with the user's profileImage value directly interpolated. An attacker who controls their profileImage can inject additional CSP directives to weaken security policy.

An attacker can set their profileImage to `; script-src 'unsafe-inline'` to weaken CSP and enable XSS execution.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/routes/userProfile.ts (L88) — sink: profileImage interpolated into CSP header
```

Evidence

routes/userProfile.ts:88:

const CSP = `img-src 'self' ${user?.profileImage}; script-src 'self' 'unsafe-eval'`

Suggested fix

Validate profileImage is a proper URL with no semicolons. Use a fixed CSP that doesn't interpolate user data.

Finding 57: NoSQL Injection via $where in Product Reviews Listing

ID	**f-116a5bf8-31b7-45b1-8e13-248fec1abc6e**
Severity	**Medium**
Risk Type	**Code Injection**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:21 PM (GMT+7)**

Description

The product reviews endpoint uses MongoDB's $where operator with string concatenation of user-supplied product ID, enabling NoSQL injection and potential server-side JavaScript execution.

An attacker can execute arbitrary JavaScript in the MongoDB context or cause denial of service via sleep().

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/routes/showProductReviews.ts (L37) — sink: MongoDB $where with string concatenation
```

Evidence

routes/showProductReviews.ts:37:

db.reviewsCollection.find({ $where: 'this.product == ' + id })

Suggested fix

Use standard MongoDB queries: db.reviewsCollection.find({ product: parseInt(id) })

Finding 58: Passwords Exposed in URL Query String via GET-Based Password Change

ID	**f-0d135aca-4881-4221-aea8-c438e7fae6a0**
Severity	**Medium**
Risk Type	**Information Disclosure**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:23 PM (GMT+7)**

Description

The password change endpoint uses GET requests with passwords as query parameters. These are logged in web server access logs, browser history, and referrer headers.

Passwords are exposed in access logs (morgan), proxy logs, browser history, and bookmark managers.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/routes/changePassword.ts (L13–L14) — sink: passwords transmitted as GET query parameters
```

Evidence

routes/changePassword.ts:13-14:

const currentPassword: [REDACTED] as string

const newPassword: [REDACTED] as string

Suggested fix

Use POST method with passwords in the request body, never in URLs.

Finding 59: No Rate Limiting on Login Endpoint Enables Brute Force

ID	**f-080755d8-3dbc-474c-9fc8-247db04cdd62**
Severity	**Medium**
Risk Type	**Authentication Bypass**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:25 PM (GMT+7)**

Description

The login endpoint (POST /rest/user/login) has no rate limiting middleware applied, allowing unlimited brute force attempts against user accounts.

An attacker can perform unlimited password guessing attempts against any account.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/routes/login.ts (L19) — gap: no rate limiting on login endpoint
```

Evidence

POST /rest/user/login has no rate limiting middleware applied

Suggested fix

Add rate limiting middleware to the login endpoint. Implement account lockout after failed attempts.

Finding 60: Denial of Service via XML Entity Expansion and YAML Bomb

ID	**f-4be52c01-8b9d-4aca-8c89-45b4e6b4fdfe**
Severity	**Medium**
Risk Type	**Denial Of Service**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:25 PM (GMT+7)**

Description

XML parsing with DTDLOAD enabled allows entity expansion attacks (Billion Laughs). YAML processing is similarly vulnerable to YAML bomb attacks using nested anchors that cause exponential memory consumption.

An attacker can cause denial of service by uploading crafted XML/YAML files that trigger exponential entity/anchor expansion, consuming all server memory.

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/lib/xml.ts (L34–L38) — sink: XML parsing with entity expansion enabled
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/routes/fileUpload.ts (L102–L121) — sink: YAML processing vulnerable to YAML bomb
```

Evidence

lib/xml.ts:34: XML_PARSE_NOENT | XML_PARSE_DTDLOAD enables entity expansion

routes/fileUpload.ts:102: YAML processing vulnerable to nested anchor bombs

Suggested fix

Disable DTD loading for XML. Set entity expansion limits. Use yaml.safeLoad() for YAML. Add file size limits.

Finding 61: [Requirement: authentication/JWT/crypto] Hardcoded JWT Private Key with Algorithm Confusion Enables Token Forgery

ID	**f-0ceb8f10-fd4a-4001-a151-9932961e0894**
Severity	**Critical**
Risk Type	**Json Web Token Vulnerabilities**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:14 PM (GMT+7)**

Description

The application uses a hardcoded RSA private key for JWT signing (lib/insecurity.ts:22-28) and relies on express-jwt@0.1.3 (package.json:112) for token verification. The express-jwt version 0.1.3 does not support algorithm enforcement (no `algorithms` option), meaning it cannot reject tokens with `alg: "none"` or HS256 algorithm confusion attacks. Additionally, passwords are hashed with unsalted MD5 (lib/insecurity.ts:41), the RSA key is only 1024-bit (below minimum 2048-bit), and the HMAC secret is hardcoded (`pa4qacea4VK9t9nGv7yZtwmj` at lib/insecurity.ts:42). No audience or issuer fields are set when signing tokens (lib/insecurity.ts:52) or verifying them (lib/insecurity.ts:186-196).

An attacker can forge arbitrary JWT tokens for any user (including admin) by using the hardcoded private key visible in the source code. Combined with the algorithm confusion vulnerability, an attacker can also craft tokens using `alg:none` or HS256 with the public key. This enables complete authentication bypass, impersonation of any user including admin, and full application takeover. The MD5 password hashing means any leaked hashes can be trivially reversed.

Verified: The private key is hardcoded in source at lib/insecurity.ts:22-28 (not loaded from env or secret store). express-jwt version confirmed as 0.1.3 in package.json:112. The `isAuthorized()` function at lib/insecurity.ts:49 only passes `{ secret: [REDACTED] }` with no algorithm restriction. The `authorize()` function at line 52 signs with RS256 but sets no `aud` or `iss`. The `updateAuthenticatedUsers()` at line 186 auto-registers any JWT that passes verification. No secret management service (AWS Secrets Manager, Vault, etc.) is referenced anywhere in the codebase.

Could not verify: Whether the source code repository is publicly accessible in production (it's an open-source project, so the key IS public). Whether any runtime mechanism prevents the use of the hardcoded key in deployed environments.

Security Requirement(s): ASA Base Pack/authentication-best-practices, ASA Base Pack/secret-protection-best-practices, ASA Base Pack/trusted-cryptography-best-practices, Juice Shop requirement pack/json-web-token-jwt-authorization-best-practices

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/lib/insecurity.ts (L21–L28) — sink: hardcoded RSA private key enables token forgery
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/lib/insecurity.ts (L49–L52) — sink: no algorithm enforcement in JWT verification
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/lib/insecurity.ts (L41–L42) — sink: MD5 password hashing and hardcoded HMAC secret
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/package.json (L112) — config: express-jwt 0.1.3 with known algorithm confusion
```

Evidence

Entry point: Any HTTP request to protected endpoints (e.g., GET /rest/basket/:id, server.ts:418)

’security.isAuthorized() at lib/insecurity.ts:49: expressJwt({ secret: [REDACTED] }) — no algorithms option

’express-jwt@0.1.3 (package.json:112) — cannot enforce algorithm, accepts alg:none

’Private key hardcoded at lib/insecurity.ts:22-28 — attacker can sign arbitrary tokens

’updateAuthenticatedUsers() at lib/insecurity.ts:186-196 — auto-registers forged token as valid session

’Token payload trusted for authorization decisions (appendUserId at line 175, isAccounting at line 155)

Additionally:

- MD5 hash: lib/insecurity.ts:41 — crypto.createHash('md5').update(data).digest('hex')
- No aud/iss in signing: lib/insecurity.ts:52 — jwt.sign(user, privateKey, { expiresIn: '6h', algorithm: 'RS256' })
- No aud/iss in verification: lib/insecurity.ts:186-196 — jwt.verify(token, publicKey, callback) with no options

Suggested fix

Replace the hardcoded private key with a secret management service (AWS Secrets Manager, HashiCorp Vault), upgrade to a modern version of express-jwt (>= 6.0) that enforces algorithm allowlists, and replace MD5 with bcrypt/argon2.

1. At lib/insecurity.ts:22-28, load the private key from environment variable or secret store instead of hardcoding.

2. At lib/insecurity.ts:49, upgrade express-jwt and add `{ algorithms: ['RS256'] }` to reject alg:none and HS256.

3. At lib/insecurity.ts:52, add `audience` and `issuer` fields to jwt.sign options.

4. At lib/insecurity.ts:186-196, add `audience` and `issuer` verification to jwt.verify.

5. At lib/insecurity.ts:41, replace MD5 with bcrypt: `bcrypt.hashSync(data, 12)`.

6. Rotate all credentials after deploying the fix since the current key is publicly known.

Finding 62: [Requirement: authorization-best-practices] IDOR on Basket Access and Missing Authorization on Product Modification

ID	**f-74a37138-a986-4ad3-836d-4a1c07b44808**
Severity	**High**
Risk Type	**Insecure Direct Object Reference**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:15 PM (GMT+7)**

Description

The basket retrieval endpoint GET /rest/basket/:id (routes/basket.ts:14) uses `security.isAuthorized()` middleware (server.ts:375,418) to verify the user has a valid JWT, but performs no ownership verification — it does not check that the authenticated user's basket ID matches the requested `:id` parameter. At routes/basket.ts:18, the basket is fetched using only `{ where: { id } }` with the user-supplied ID. The challenge verification code at line 20-22 confirms this is an intentional IDOR — it checks if `user.bid != parseInt(id)` to solve a challenge, but doesn't block access.

Any authenticated user can access any other user's shopping basket by simply changing the numeric ID in the URL (e.g., GET /rest/basket/1, /rest/basket/2, etc.). Baskets contain product selections, pricing, and coupon data that reveals shopping behavior. Combined with unauthenticated product modification (PUT /api/Products/:id has auth commented out at server.ts:389), the application has systemic authorization failures.

Verified: No ownership check exists in routes/basket.ts — the query is `BasketModel.findOne({ where: { id } })` with only the URL parameter. The challenge code at line 20-22 detects the IDOR but does not prevent it. server.ts:389 confirms PUT /api/Products/:id auth is commented out (`// app.put('/api/Products/:id', security.isAuthorized())`).

Could not verify: Whether middleware in front of the application performs additional authorization checks. No such middleware was found in the codebase.

Security Requirement(s): ASA Base Pack/authorization-best-practices

Code locations

`/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/routes/basket.ts (L14–L28) — sink: IDOR - basket access without ownership check`

`/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/server.ts (L418) — config: isAuthorized only checks JWT validity, not ownership`

`/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/server.ts (L389) — config: PUT /api/Products/:id auth commented out`

Evidence

Entry point: GET /rest/basket/:id (server.ts:418 — security.isAuthorized() but no ownership check)

’routes/basket.ts:14 — retrieveBasket()

’const id = req.params.id (line 17 — user-controlled)

’BasketModel.findOne({ where: { id } }) (line 18 — no ownership filter)

’Challenge code at line 20-22 confirms bug: checks user.bid != parseInt(id) but does NOT block access

’res.json(utils.queryResultToJson(basket)) (line 28 — returns any basket)

Also: PUT /api/Products/:id — server.ts:389 has auth commented out:

// app.put('/api/Products/:id', security.isAuthorized())

Suggested fix

Add ownership verification to the basket retrieval endpoint and uncomment/add auth to the product PUT endpoint.

1. At routes/basket.ts:18, change the query to include the user's basket ID: `BasketModel.findOne({ where: { id, UserId: authenticatedUser.data.id } })` or verify `authenticatedUser.bid == id` before proceeding.
2. At server.ts:389, uncomment the authorization: `app.put('/api/Products/:id', security.isAuthorized())` and add an admin-only check since regular users shouldn't modify products.
3. Apply the same ownership check pattern to all resources: reviews (routes/createProductReviews.ts, routes/updateProductReviews.ts), addresses, and complaints.

Finding 63: [Requirement: information-protection] Credit Card Numbers Stored as Plaintext Integers Extractable via SQL Injection

ID	**f-3d7880bd-78fc-4fc8-91c3-6985cc97a8fc**
Severity	**High**
Risk Type	**Cryptographic Vulnerabilities**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:15 PM (GMT+7)**

Description

Credit card numbers are stored as plaintext integers in the database (models/card.ts:38-44, type: DataTypes.INTEGER). The `cardNum` field has integer validation with min/max constraints (lines 39-43) confirming it stores the full 16-digit card number without encryption or tokenization. While the payment display endpoint masks cards for API responses (routes/payment.ts:31-32), the raw numbers persist in the SQLite database. This data is extractable via the SQL injection vulnerability in the login endpoint (routes/login.ts:37) or product search (routes/search.ts:24).

An attacker who exploits any SQL injection vulnerability can extract all stored credit card numbers in plaintext via a UNION SELECT from the Cards table. Even without SQL injection, database file access (possible via path traversal or server compromise) would yield all card numbers. This violates PCI-DSS requirements and the information protection security requirement.

Verified: models/card.ts:38-44 confirms `cardNum: { type: DataTypes.INTEGER }` — no encryption layer. routes/payment.ts:31-32 only masks for display (`'*'.repeat(12) + cardNumber.substring(cardNumber.length - 4)`) but the DB retains the full number. SQL injection at routes/login.ts:37 and routes/search.ts:24 could extract this data via UNION injection. No encryption-at-rest configuration exists for the SQLite database.

Could not verify: Whether application-level encryption middleware intercepts writes/reads from the Card model (none was found). Whether disk-level encryption protects the SQLite file at rest in deployment.

Security Requirement(s): ASA Base Pack/information-protection-best-practices

Code locations

`/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/models/card.ts (L38–L44) — sink: credit card numbers stored as plaintext integers`

`/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/routes/payment.ts (L31–L32) — evidence: raw card numbers read from DB then masked only for display`

`/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/routes/search.ts (L24) — exploitation: SQL injection enables extraction of all card numbers`

Evidence

models/card.ts:38-44:

cardNum: {

type: DataTypes.INTEGER,

validate: {

isInt: true,

min: 1000000000000000,

max: 9999999999999998

}

}

’Full 16-digit card number stored as plaintext integer

routes/payment.ts:31-32 (masking only for display):

const cardNumber = String(card.cardNum)

displayableCard.cardNum = '*'.repeat(12) + cardNumber.substring(cardNumber.length - 4)

’Raw number read from DB, masked only in response

Extraction path via SQL injection:

routes/search.ts:24: `SELECT * FROM Products WHERE ((name LIKE '%${criteria}%'...`

’Attacker injects: `')) UNION SELECT id,UserId,fullName,cardNum,expMonth,expYear,null,null,null FROM Cards--`

Suggested fix

Never store full credit card numbers. Use tokenization or store only the last 4 digits.

1. At models/card.ts:38, change cardNum to store only a masked/tokenized reference: `cardNum: { type: DataTypes.STRING }` containing only the last 4 digits.

2. Use a payment processor (Stripe, Braintree) that handles card storage — the application should only store

a payment token reference.

3. If full numbers must be stored temporarily, use application-level encryption with a key managed via KMS, and ensure the encrypted values cannot be extracted via SQL injection (encrypt before write, decrypt only in authorized code paths).

4. Fix the SQL injection vulnerabilities at routes/login.ts:37 and routes/search.ts:24 using parameterized queries.

Finding 64: [Requirement: secure-by-default] Insecure Default Configuration: Permissive CORS, No CSP, Insecure Cookies, Public Directories

ID	**f-15de3835-8af1-4821-9a12-41c84a9d7985**
Severity	**High**
Risk Type	**Security Misconfiguration**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:15 PM (GMT+7)**

Description

The application's default configuration is insecure across multiple dimensions: (1) Permissive CORS allowing all origins via `app.options('*', cors())` and `app.use(cors())` at server.ts:182-183, (2) XSS filter explicitly disabled — helmet's xssFilter is NOT used while noSniff/frameguard are (server.ts:186-187), (3) Public directory browsing enabled for sensitive directories (ftp/, encryptionkeys/, logs/ at server.ts:288-302), (4) `trust proxy` enabled at server.ts:361 which allows X-Forwarded-For spoofing to bypass rate limits (server.ts:365), and (5) JWT cookie set without httpOnly, secure, or sameSite attributes (lib/insecurity.ts:192 — `res.cookie('token', token)` with no options).

These insecure defaults combine to create a permissive attack surface: CORS allows cross-origin credential theft, the absent XSS filter removes a browser defense layer, public directory browsing leaks sensitive files, trust proxy enables rate limit bypass, and insecure cookie settings enable session theft via XSS.

Verified: server.ts:182-183 confirms `cors()` with no origin restriction. server.ts:186-187 only uses `helmet.noSniff()` and `helmet.frameguard()` — no xssFilter or CSP. server.ts:288-302 confirms public directories. server.ts:361 confirms `trust proxy`. lib/insecurity.ts:192 confirms `res.cookie('token', token)` with no httpOnly/secure/sameSite. No Content-Security-Policy header is set globally.

Could not verify: Whether a reverse proxy adds CSP, restricts CORS, or forces secure cookie attributes in deployment. No such configuration was found in the infrastructure code.

Security Requirement(s): ASA Base Pack/secure-by-default-best-practices

Code locations

`/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/server.ts (L182–L187) — sink: permissive CORS and missing XSS/CSP headers`

`/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/server.ts (L288–L302) — sink: public directory browsing for sensitive directories`

`/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/lib/insecurity.ts (L192) — sink: cookie without httpOnly/secure/sameSite flags`

`/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/server.ts (L361) — config: trust proxy enables rate limit bypass via X-Forwarded-For`

Evidence

server.ts:182-183:

app.options('*', cors())

app.use(cors()) // No origin restriction

server.ts:186-187:

app.use(helmet.noSniff())

app.use(helmet.frameguard())

// Note: NO xssFilter(), NO contentSecurityPolicy()

server.ts:288-302:

app.use('/ftp', serveIndexMiddleware, serveIndex('ftp', ...)) // Public browsing

app.use('/encryptionkeys', serveIndexMiddleware, serveIndex(...)) // Keys exposed

app.use('/support/logs', serveIndexMiddleware, serveIndex(...)) // Logs exposed

server.ts:361:

app.enable('trust proxy') // Enables X-Forwarded-For spoofing

lib/insecurity.ts:192:

res.cookie('token', token) // No httpOnly, secure, or sameSite

Suggested fix

Restrict CORS, add CSP, set cookie security flags, remove public directory browsing, and fix trust proxy handling.

1. At server.ts:182-183, configure CORS with specific allowed origins: `cors({ origin: ['https://your-domain.com'], credentials: true })`.
2. Add `helmet.contentSecurityPolicy()` with a restrictive policy (no unsafe-eval, no unsafe-inline).
3. At lib/insecurity.ts:192, set cookie with security flags: `res.cookie('token', token, { httpOnly: true, secure: true, sameSite: 'strict' })`.

4. At server.ts:288-302, remove or authenticate the directory browsing endpoints.

5. At server.ts:361, either disable trust proxy or ensure rate limit keyGenerator validates the X-Forwarded-For chain against trusted proxy IPs.

Finding 65: [Requirement: log-protection] Logs Publicly Accessible Without Authentication Containing Sensitive Data

ID	**f-1e381a8e-2c4a-43c1-8707-5c325bad18dd**
Severity	**High**
Risk Type	**Information Disclosure**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:16 PM (GMT+7)**

Description

Log files are served without authentication at `/support/logs` (server.ts:300-302) via directory browsing (serveIndex) and direct file access (routes/logfileServer.ts:10-14). The logs contain sensitive data including passwords from GET-based password change URLs (logged by morgan's 'combined' format at server.ts:357). No redaction of PII, credentials, or secrets is applied to log entries. The morgan 'combined' format logs the full request URL, user agent, referrer, and IP address.

An unauthenticated attacker can access all application logs, which contain: (1) full URLs with passwords from password change operations, (2) user IP addresses, (3) user agents and session information, (4) API paths revealing application structure. This enables direct credential theft and user tracking.

Verified: server.ts:300 uses `serveIndex('logs', ...)` with no auth middleware preceding it. server.ts:302 routes to `serveLogFiles()` which at routes/logfileServer.ts:14 calls `res.sendFile(path.resolve('logs/', file))` with no auth check beyond preventing forward slashes. Morgan 'combined' format (server.ts:357) includes full request URL. No log redaction is configured.

Could not verify: Whether the /support/logs path is blocked by a reverse proxy or WAF at the infrastructure level.

Security Requirement(s): ASA Base Pack/log-protection-best-practices

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/server.ts (L300–L302) — sink: unauthenticated log serving with sensitive data
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/routes/logfileServer.ts (L10–L14) — sink: serves log files without authentication
```

`/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/server.ts (L357) — source: morgan logs full URLs including passwords`

Evidence

server.ts:300-302:

app.use('/support/logs', serveIndexMiddleware, serveIndex('logs', { icons: true, view: 'details' }))

app.use('/support/logs', verify.accessControlChallenges()) // Only challenge verification, NOT access control

app.use('/support/logs/:file', serveLogFiles())

routes/logfileServer.ts:10-14:

export function serveLogFiles () {

return ({ params }, res, next) => {

const file = params.file

if (!file.includes('/')) {

res.sendFile(path.resolve('logs/', file)) // No authentication

}

}

}

server.ts:357:

app.use(morgan('combined', { stream: accessLogStream }))

// combined format logs full URLs including query parameters with passwords

Suggested fix

Restrict log access to authenticated administrators and implement log redaction.

1. At server.ts:300-302, add `security.isAuthorized()` and admin role check before the log serving routes.
2. Configure morgan with a custom format that redacts query parameters from logged URLs, or use a format that excludes the query string.
3. Implement log redaction to mask/remove PII, passwords, and tokens before writing to log files.
4. Set appropriate retention policies and ensure logs are stored in a tamper-resistant location.

Finding 66: [Requirement: privileged-access] Role Injection in User Registration Enables Admin Self-Enrollment Without Guardrails

ID	**f-a8663329-d201-46ab-a583-746630f50f8c**
Severity	**High**
Risk Type	**Privilege Escalation**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:15 PM (GMT+7)**

Description

Privileged access lacks adequate guardrails: (1) User registration at POST /api/Users (server.ts:427-439) accepts a `role` field in the request body without blocking role injection — the `verify.registerAdminChallenge()` middleware at server.ts:437 detects the attempt for challenge-solving purposes but does not prevent it, allowing self-registration as admin. (2) The application configuration endpoint GET /rest/admin/application-configuration (server.ts:626) is accessible without authentication, exposing chatbot configuration, challenge settings, and internal application details. (3) No audit logging exists for any administrative actions.

An attacker can register a new admin account by sending `POST /api/Users` with `{"email":"attacker@evil.com", "password":"pass", "passwordRepeat":"pass", "role":"admin"}`. This bypasses the need to compromise existing admin credentials. The unauthenticated config endpoint exposes internal application structure. Combined with the lack of audit logging, administrative abuse would go undetected.

Verified: server.ts:427-439 shows the user creation middleware chain — `verify.registerAdminChallenge()` at line 437 only checks for the challenge, does not reject the request. The finale-rest auto-generated POST /api/Users endpoint accepts all model fields including `role`. GET /rest/admin/application-configuration at server.ts:626 has no auth middleware. No admin action logging exists anywhere.

Could not verify: Whether the User model has a beforeCreate hook that strips the role field (checked — model allows role via DataTypes). Whether a proxy strips role from POST body (none found).

Security Requirement(s): ASA Base Pack/privileged-access-best-practices

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/server.ts (L427–L439) — sink: user registration accepts role field enabling admin
self-registration
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/server.ts (L626) — sink: unauthenticated admin configuration endpoint
```

Evidence

server.ts:427-437 (user registration chain):

app.post('/api/Users', (req, res, next) => {

// Only trims email/password, does NOT strip 'role' field

next()

})

app.post('/api/Users', verify.registerAdminChallenge()) // Detects but doesn't block

app.post('/api/Users', verify.passwordRepeatChallenge())

app.post('/api/Users', verify.emptyUserRegistration())

'finale-rest creates the user with all submitted fields including role

server.ts:626:

app.get('/rest/admin/application-configuration', utils.asyncHandler(retrieveAppConfiguration()))

'No security.isAuthorized() middleware

Suggested fix

Strip the role field from user registration requests and add authentication to admin endpoints.

1. At server.ts:427 in the POST /api/Users middleware chain, add: `delete req.body.role` to strip the role field before it reaches finale-rest.
2. At server.ts:626, add `security.isAuthorized()` middleware to the application-configuration endpoint, and restrict it to admin role.
3. Add audit logging for all admin-level actions (user creation with elevated roles, configuration access).
4. Implement role-based access control that prevents self-elevation — role changes should only be allowed by existing admins through a separate admin endpoint.

Finding 67: [Requirement: log-protection/information-protection] Password Change via GET Logged by Morgan and Served Publicly Creates Credential Exposure Chain

ID	**f-948aba49-0a81-4023-a394-2b2a543a4bfd**
Severity	**High**
Risk Type	**Information Disclosure**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:14 PM (GMT+7)**

Description

A multi-stage credential exposure chain exists: (1) passwords are sent as GET query parameters in the password change endpoint (routes/changePassword.ts:13-14 reads `query.current`, `query.new`, `query.repeat`), (2) morgan's 'combined' format logs the full URL including query strings to the access log file (server.ts:357), and (3) the log files are served without authentication at `/support/logs/:file` (routes/logfileServer.ts:10-14, server.ts:300-302 with directory listing).

An unauthenticated attacker can browse `/support/logs` to discover log filenames, then download them to extract plaintext passwords from URLs like `GET /rest/user/change-password?current=secret&new=newsecret&repeat=newsecret`. This gives the attacker credentials for any user who has changed their password, enabling account takeover at scale.

Verified: changePassword uses GET with query params (routes/changePassword.ts:13). Morgan 'combined' format includes full URL with query string (server.ts:357). Log files served at /support/logs without auth (server.ts:300-302, routes/logfileServer.ts:14 only checks for forward slashes). No redaction of sensitive URL parameters exists in the morgan configuration.

Could not verify: Whether a WAF strips query parameters from log entries, whether a reverse proxy truncates logged URLs, or whether network access to /support/logs is restricted at infrastructure level.

Security Requirement(s): ASA Base Pack/log-protection-best-practices, ASA Base Pack/information-protection-best-practices, ASA Base Pack/secret-protection-best-practices

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/routes/logfileServer.ts (L10–L14) — sink: unauthenticated log file serving exposing
passwords
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/routes/changePassword.ts (L13–L14) — source: passwords transmitted as GET query
parameters
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/server.ts (L357) — intermediary: morgan combined format logs full URLs with passwords
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/server.ts (L300–L302) — intermediary: unauthenticated directory listing of log files
```

Evidence

Stage 1 - Passwords in URLs:

GET /rest/user/change-password?current=X&new=Y&repeat=Y (routes/changePassword.ts:13-14)

’query.current, query.new, query.repeat read from URL query string

Stage 2 - Logging:

’server.ts:357: app.use(morgan('combined', { stream: accessLogStream }))

’Combined format logs: IP - - [date] "GET /rest/user/change-password?current=X&new=Y&repeat=Y

HTTP/1.1" status ...

Stage 3 - Unauthenticated log access:

’server.ts:300: app.use('/support/logs', serveIndexMiddleware, serveIndex('logs', ...))

’server.ts:302: app.use('/support/logs/:file', serveLogFiles())

’routes/logfileServer.ts:14: res.sendFile(path.resolve('logs/', file)) — no auth check

Suggested fix

Change the password change endpoint from GET to POST, add log redaction for sensitive parameters, and require authentication for log file access.

1. At routes/changePassword.ts:13, change from GET to POST and read parameters from req.body instead of req.query.
2. At server.ts:357, configure morgan to redact sensitive URL parameters or use a custom token format that excludes query strings from logging.
3. At server.ts:300-302, add authentication middleware (security.isAuthorized()) before the log file serving routes.
4. At routes/logfileServer.ts, add an authorization check to restrict log access to admin users only.

Finding 68: [Requirement: information-protection/secure-by-default] HTTP ALB Listener Forwards Traffic Without HTTPS Redirect

ID	**f-c2b457aa-1914-4446-bf2e-3c7637f35ca1**
Severity	**High**
Risk Type	**Security Misconfiguration**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:15 PM (GMT+7)**

Description

The Terraform configuration at infrastructure/terraform/networking.tf:157-166 defines an AWS ALB HTTP listener (port 80) with a `forward` action that sends traffic directly to the target group without redirecting to HTTPS. An HTTPS listener exists at lines 183-194, but the HTTP listener does not redirect to it. Combined with the password change endpoint using GET parameters (routes/changePassword.ts:13-14) and JWT tokens transmitted in Authorization headers, this allows plaintext interception of credentials.

An attacker performing a man-in-the-middle attack on the network path can intercept HTTP traffic containing JWT tokens, passwords in query strings, and payment data. Since the ALB forwards HTTP traffic rather than redirecting to HTTPS, clients connecting via HTTP will have all data transmitted in cleartext.

Verified: The HTTP listener at networking.tf:157-166 uses `type = "forward"` with `target_group_arn`. This should be `type = "redirect"` with `protocol = "HTTPS"` to enforce TLS. The HTTPS listener exists at lines 183-194 with proper TLS policy. The hardcoded TLS private key at line 171 is the same key as the JWT private key in lib/insecurity.ts:22 — meaning TLS could also be compromised.

Could not verify: Whether the application is actually deployed using this Terraform configuration, or whether additional infrastructure (CloudFront, DNS-level redirect) forces HTTPS before reaching the ALB.

Security Requirement(s): ASA Base Pack/information-protection-best-practices, ASA Base Pack/secure-by-default-best-practices

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/in-
frastructure/terraform/networking.tf (L157–L166) — sink: HTTP listener forwards traffic
without HTTPS redirect
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/in-
frastructure/terraform/networking.tf (L183–L194) — comparison: HTTPS listener exists but
HTTP not redirected to it
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/routes/changePassword.ts (L13–L14) — impact: passwords in query strings sent over
unencrypted HTTP
```

Evidence

infrastructure/terraform/networking.tf:157-166:

resource "aws_lb_listener" "http" {

load_balancer_arn = aws_lb.juice_shop.arn

port = 80

protocol = "HTTP"

default_action {

type = "forward" • Should be "redirect" to HTTPS

target_group_arn = aws_lb_target_group.juice_shop.arn

}

}

Compared to proper HTTPS listener at lines 183-194 with ssl_policy = "ELBSecurityPoli-

cy-TLS13-1-2-2021-06"

Impact amplified by:

’routes/changePassword.ts:13-14 — passwords in GET query strings over HTTP

’JWT tokens in Authorization headers over HTTP

’Same TLS private key as JWT key (networking.tf:171 = lib/insecurity.ts:22)

Suggested fix

Change the HTTP listener from forwarding to redirecting to HTTPS.

1. At infrastructure/terraform/networking.tf:157-166, change the default_action:

```
default_action {
type = "redirect"
redirect {
port = "443"
```

```
    protocol    = "HTTPS"
    status_code = "HTTP_301"
  }
}
```

2. Remove the hardcoded TLS private key at line 171 and use AWS Certificate Manager (ACM) instead.

3. Change the password change endpoint from GET to POST (routes/changePassword.ts) to avoid credentials in URLs.

Finding 69: [Requirement: file-hosting/information-protection/tenant-isolation] Unauthenticated FTP-Style File Hosting Exposes Customer PII and Secrets

ID	**f-fd4ed050-9407-410e-84e4-18c2d2c60641**
Severity	**High**
Risk Type	**Information Disclosure**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:14 PM (GMT+7)**

Description

The application serves files from the `/ftp` directory with no authentication via directory browsing (server.ts:288) and file download (server.ts:289, routes/fileServer.ts:13-32). Order confirmation PDFs containing customer email addresses, order details, and prices are written to the `ftp/` directory (routes/order.ts:45) and become accessible to any unauthenticated user. Additionally, `/encryptionkeys` (server.ts:296-297) exposes cryptographic key files and `/support/logs` (server.ts:300-302) exposes log files — all without authentication or authorization.

Any unauthenticated internet user can browse the `/ftp` directory listing to discover order confirmation PDFs, then download them to access customer PII (email, order details, pricing). The `/encryptionkeys` endpoint exposes the JWT public key. The `/support/logs` endpoint exposes access logs that contain full request URLs including passwords from the GET-based password change endpoint. This violates the requirement that all file hosting must use purpose-built APIs with proper authorization, audit logging, and access governance.

Verified: No authentication middleware exists before serveIndex/servePublicFiles for `/ftp` (server.ts:288-289). The `servePublicFiles()` function (routes/fileServer.ts:13) only checks for forward slashes and file extensions — no auth. Order PDFs are confirmed written to `ftp/` at routes/order.ts:45 via `fs.createWriteStream(path.join('ftp/', pdfFile))`. The file extension allowlist at routes/fileServer.ts:25 explicitly allows `.pdf`. No access log or audit trail is generated for file downloads.

Could not verify: Whether a reverse proxy or WAF in front of the application blocks access to `/ftp`, `/encryptionkeys`, or `/support/logs`. Whether the ftp/ directory contains actual customer order PDFs in a production deployment (it does based on the code path from order placement).

Security Requirement(s): Juice Shop requirement pack/forbid-ftp-and-sftp-file-hosting-from-company-applications, ASA Base Pack/information-protection-best-practices, ASA Base Pack/tenant-isolation-best-practices, ASA Base Pack/authorization-best-practices, ASA Base Pack/audit-logging-best-practices

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/server.ts (L288–L290) — sink: unauthenticated FTP-style directory listing and file hosting
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/routes/fileServer.ts (L13–L32) — sink: serves files without authentication, allows .pdf
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/routes/order.ts (L41–L45) — source: order PDFs with customer PII written to ftp/
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/server.ts (L300–L302) — sink: unauthenticated log file serving
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/server.ts (L296–L297) — sink: unauthenticated encryption key hosting
```

Evidence

Entry point: GET /ftp (unauthenticated, server.ts:288)

’serveIndex('ftp', { icons: true }) — directory listing of all files in ftp/

’GET /ftp/:file (server.ts:289) ’ servePublicFiles() (routes/fileServer.ts:13)

’endsWithAllowlistedFileType() checks .md or .pdf only (line 25) — .pdf is allowed

’res.sendFile(path.resolve('ftp/', file)) at line 31 — serves file without auth

Source of sensitive files:

’routes/order.ts:45: fs.createWriteStream(path.join('ftp/', pdfFile))

’PDF contains: customer email (line 64), order ID (line 65), product details, prices

Similarly:

’GET /encryptionkeys/:file (server.ts:297) ’ routes/keyServer.ts — no auth

’GET /support/logs/:file (server.ts:302) ’ routes/logfileServer.ts:14 — no auth

Suggested fix

Remove the public directory browsing and static file serving for /ftp, /encryptionkeys, and /support/logs. Store order PDFs in a location not served publicly and provide authenticated download endpoints. Use purpose-built APIs with proper authorization for all file access.

1. At server.ts:288-290, remove the serveIndex and servePublicFiles middleware for /ftp.

2. At server.ts:296-297, remove the public encryptionkeys directory listing.

3. At server.ts:300-302, remove the public logs directory listing or add authentication middleware.

4. At routes/order.ts:45, write PDFs to a non-publicly-served directory (e.g., `data/orders/`).

5. Create authenticated API endpoints for order PDF downloads that verify the requesting user owns the order.

Finding 70: [Requirement: audit-logging/privileged-access] Complete Absence of Security Audit Logging Across All Security-Critical Operations

ID	**f-2aceb40a-770e-4a50-ac59-817a49628865**
Severity	**Medium**
Risk Type	**Security Misconfiguration**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:15 PM (GMT+7)**

Description

The application completely lacks security event audit logging. Authentication events (login success/failure at routes/login.ts:32-54), password changes (routes/changePassword.ts:44-56), 2FA setup/disable (routes/2fa.ts:97-137), wallet transactions (routes/wallet.ts:21-35), and administrative actions are not logged with security context. The only logging is generic HTTP access logging via morgan at server.ts:357, which logs URLs and status codes but lacks structured security event data (user ID, action type, success/failure, IP attribution).

The lib/logger.ts file (lines 1-12) only configures a basic winston console logger with no security event formatters, no structured event fields, and no dedicated security log stream. This makes incident investigation impossible — there is no way to determine who logged in, when passwords were changed, or what administrative actions occurred.

Verified: Searched all route handlers for logging statements — found none in login.ts, changePassword.ts, 2fa.ts, wallet.ts, or any admin-related handlers. The only log calls are in the chatbot (routes/chat.ts — logger.warn for errors only) and startup code. No security information and event management (SIEM) integration exists. The morgan configuration (server.ts:349-357) only uses 'combined' format which is a generic access log, not a security event log.

Could not verify: Whether an external logging agent (CloudWatch, Datadog) captures application output and provides security monitoring. No integration with such services was found in the codebase or Terraform configuration.

Security Requirement(s): ASA Base Pack/audit-logging-best-practices, ASA Base Pack/privileged-access-best-practices, ASA Base Pack/log-protection-best-practices

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/routes/login.ts (L32–L54) — gap: no audit logging of authentication attempts
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/routes/changePassword.ts (L44–L56) — gap: no audit logging of password changes
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/routes/2fa.ts (L97–L137) — gap: no audit logging of 2FA state changes
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/routes/wallet.ts (L21–L35) — gap: no audit logging of financial transactions
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/lib/logger.ts (L1–L12) — infrastructure: basic logger with no security event support
```

Evidence

routes/login.ts:32-54 — Login handler performs authentication but:

’No log of successful login (user ID, IP, timestamp)

’No log of failed login (attempted email, IP, timestamp)

’Only challenge verification code exists

routes/changePassword.ts:44-56 — Password change handler:

’No log of password change (user ID, IP, success/failure)

routes/2fa.ts:97-137 — 2FA setup/disable:

’No log of 2FA state changes (user ID, action, timestamp)

routes/wallet.ts:21-35 — Financial transactions:

’No log of wallet balance changes (user ID, amount, timestamp)

lib/logger.ts:1-12 — Only basic winston console logger:

’No security event support, no structured fields, no dedicated transport

server.ts:357 — Only generic access log:

’app.use(morgan('combined', { stream: accessLogStream }))

Suggested fix

Implement structured security event logging for all security-critical operations.

1. Create a security audit logger module that outputs structured JSON events with fields: timestamp,

event_type, user_id, source_ip, outcome (success/failure), resource, details.

2. At routes/login.ts:32 (success) and the 401 branch (failure), log authentication events.

3. At routes/changePassword.ts:51, log password changes with user ID and success status.

4. At routes/2fa.ts:97 and :118, log 2FA state changes.

5. At routes/wallet.ts:25, log financial transactions with user ID and amount.

6. Send security events to a centralized, tamper-resistant log store (CloudWatch Logs, Splunk, etc.) separate from the application's access logs.

Finding 71: [Requirement: authentication/secure-by-default] No JWT Token Revocation Enables Persistent Access After Password Change

ID	**f-6bf4e1cf-6579-453d-a147-ce67e0765052**
Severity	**Medium**
Risk Type	**Session Token Vulnerabilities**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:16 PM (GMT+7)**

Description

No JWT token revocation mechanism exists in the application. The `authenticatedUsers` map (lib/insecurity.ts:70-91) has `put()` and `get()` methods but no `remove()` or `delete()` method. Password changes (routes/changePassword.ts:44-56) update the password in the database but do not invalidate the existing JWT token stored in the map. Data erasure (routes/dataErasure.ts:88) only clears the cookie but does not remove the token from the server-side session map. There is no logout endpoint that removes tokens.

A stolen JWT token remains valid for the full 6-hour expiry (lib/insecurity.ts:52 — `expiresIn: '6h'`) regardless of password changes, account deletion, or data erasure requests. An attacker who obtains a token (via XSS, log exposure, or network interception) retains access even after the user changes their password. The session map grows indefinitely with no cleanup.

Verified: lib/insecurity.ts:70-91 shows the authenticatedUsers object with only `put`, `get`, `tokenOf`, `from`, `updateFrom` methods — no removal. routes/changePassword.ts:44-56 calls `user.update({ password: [REDACTED] })` but doesn't touch authenticatedUsers. routes/dataErasure.ts:88 only does `res.clearCookie('token')`. No logout or token invalidation endpoint exists in server.ts or any route file.

Could not verify: Whether a deployment-level token blacklist exists outside the application code (none referenced).

Security Requirement(s): ASA Base Pack/authentication-best-practices, ASA Base Pack/secure-by-default-best-practices

Code locations

`/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/lib/insecurity.ts (L70–L91) — sink: authenticatedUsers map has no token removal mechanism`

`/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/routes/changePassword.ts (L44–L56) — gap: password change does not invalidate existing sessions`

`/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-master/routes/dataErasure.ts (L88) — gap: data erasure only clears cookie, not server-side token`

Evidence

lib/insecurity.ts:70-91 (authenticatedUsers — no remove method):

```
export const authenticatedUsers: IAuthenticatedUsers = {
 tokenMap: {},
 idMap: {},
 put: function (token, user) { this.tokenMap[token: [REDACTED]; this.idMap[user.data.id] = token },
 get: function (token) { return token ? this.tokenMap[utils.unquote(token)] : undefined },
 tokenOf: function (user) { return user ? this.idMap[user.id] : undefined },
 from: function (req) { ... },
 updateFrom: function (req, user) { ... }
// NO remove/delete/invalidate method
}
```

routes/changePassword.ts:51 (no token invalidation):

```
await user.update({ password: [REDACTED] })
// authenticatedUsers still contains the old token ’ old token still works
```

routes/dataErasure.ts:88 (cookie clear only):

```
res.clearCookie('token')
// Token still in authenticatedUsers map, Bearer header auth still works
```

lib/insecurity.ts:52 (6-hour expiry):

```
jwt.sign(user, privateKey, { expiresIn: '6h', algorithm: 'RS256' })
```

Suggested fix

Implement token revocation on security-critical actions and add a logout endpoint.

1. Add a `remove(token)` method to the authenticatedUsers object at lib/insecurity.ts:70 that deletes the token from both tokenMap and idMap.

2. At routes/changePassword.ts:51, after password update, call `authenticatedUsers.remove(currentToken)` to invalidate the current session, forcing re-authentication.

3. At routes/dataErasure.ts:88, call `authenticatedUsers.remove(token)` in addition to clearing the cookie.

4. Create a POST /rest/user/logout endpoint that removes the token from authenticatedUsers.

5. Consider shorter token expiry (e.g., 15-30 minutes) with a refresh token mechanism.

Finding 72: [Requirement: JWT/authorization/tenant-isolation] Chatbot Uses Unverified JWT Decode Enabling Cross-User Order Data Access

ID	**f-c29cc6f6-1f1c-4e3b-8038-6fbb260784b7**
Severity	**Medium**
Risk Type	**Insecure Direct Object Reference**
Confidence	**High**
Status	**Active**
Identified on	**8/13/2026, 4:50:16 PM (GMT+7)**

Description

The chatbot order lookup at routes/chat.ts:42-46 uses `security.decode(token)` to extract the user ID from the JWT for authorization decisions. The `decode()` function at lib/insecurity.ts:56 only performs base64 decoding of the JWT payload (`jws.decode(token)?.payload`) WITHOUT signature verification. This means an attacker can craft an arbitrary JWT with any user ID in the payload, and the chatbot will trust it for order lookups.

The `getOrderById` tool at routes/chat.ts:158-171 retrieves orders based on the unverified user ID. An attacker can forge a JWT with another user's ID to access their order history through the chatbot, accessing delivery addresses, purchase details, and email information. This is particularly concerning because the chat endpoint at POST /rest/chat (server.ts:657) has no authentication middleware at all.

Verified: routes/chat.ts:42-46 `getUserId()` calls `security.decode(token)` at lib/insecurity.ts:56 which is `jws.decode(token)?.payload` — no verification. The `getOrderById` tool at line 158 calls `getUserId(req)` and uses the result to filter orders. POST /rest/chat at server.ts:657 has no auth middleware (-`app.post('/rest/chat', utils.asyncHandler(chat()))`). The `utils.jwtFrom(req)` at lib/utils.ts:117 reads from Authorization header.

Could not verify: Whether the LLM provider's tool-calling behavior can be reliably triggered by an attacker to invoke getOrderById with a specific order ID. The attacker would need to craft a message that causes the LLM to call the tool.

Security Requirement(s): ASA Base Pack/authentication-best-practices, ASA Base Pack/authorization-best-practices, ASA Base Pack/tenant-isolation-best-practices, Juice Shop requirement pack/json-web-token-jwt-authorization-best-practices

Code locations

```
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/routes/chat.ts (L42–L46) — sink: getUserId uses unverified JWT decode for
authorization
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/lib/insecurity.ts (L56) — source: decode() does not verify JWT signature
/tmp/opt/source_code/code_sc-8c9626ec-f3a8-4920-9c83-21cfc2df18f2/juice-shop-mas-
ter/routes/chat.ts (L158–L172) — impact: getOrderById returns order details based on
unverified identity
```

Evidence

routes/chat.ts:42-46 (getUserId uses unverified decode):

```
export async function getUserId (req: Request): Promise<number | undefined> {
 const token: [REDACTED]
 if (!token) return undefined
 const decoded = security.decode(token) as { data?: { id?: number } } | undefined
 return decoded?.data?.id
}
```

lib/insecurity.ts:56 (decode without verify):

```
export const decode = (token: [REDACTED] => { return jws.decode(token)?.payload }
```

routes/chat.ts:158-171 (getOrderById trusts unverified userId):

```
execute: async ({ orderId }) => {
 const userId = await getUserId(req)  // Uses unverified decode
 if (!userId) return { error: 'Customer not authenticated' }
 const user = await UserModel.findByPk(userId, { attributes: ['email'] })
...
 const order = await db.ordersCollection.findOne({ orderId })
 if (order.email !== maskedEmail) return { error: 'Order does not belong to the current customer' }
 return order
}
```

POST /rest/chat (server.ts:657 — no auth middleware):

```
app.post('/rest/chat', utils.asyncHandler(chat()))
```

Suggested fix

Use verified JWT decoding (security.verify + security.decode) or the authenticatedUsers map for identity in the chatbot.

1. At routes/chat.ts:42-46, replace `security.decode(token)` with `security.verify(token) && security.decode(token)` to ensure signature verification before trusting the user ID.
2. Alternatively, use `security.authenticatedUsers.from(req)` which looks up verified tokens in the session map.
3. At server.ts:657, add `security.isAuthorized()` middleware to the chat endpoint to reject unauthenticated requests at the route level.